

SOLIDWORKS API (.NET VBA C#) — Examples (10/50): Create_Command_Manager_Tab_and_Tab_Boxes - Create_Drawing_Sheet_Zones_Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Create CommandManager Tab and Tab Boxes Example (C#)

This example shows how to create an add-in that creates a CommandManager tab and tab boxes.

NOTES:

- This sample code is part of a project that was built using the C# Add-In Wizard.
- To run this sample code, you need to create a complete C# project.
 1. Copy this sample code into **SwAddin.cs** of the new project.
 2. Add references as needed to debug and run the program.
- Your add-in must check to see if the tab already exists. If the tab already exists, then you should use that tab instead of creating a new tab. If your add-in does not check for an existing tab, then SOLIDWORKS Design will create a new tab each time it starts up.
- Users can add buttons to and remove buttons from your CommandManager tab. However, if your add-in removes the CommandManager tab upon exiting SOLIDWORKS Design, then any user customizations will be lost.

\\-----

\\ SwAddin.cs

```
using System;
using System.Diagnostics;
using System.Collections;
using System.Reflection;
using System.Runtime.InteropServices;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using SolidWorksTools;
using SolidWorksTools.File;
namespace SwCSharpAddin1
{
    [Guid("c5ede50f-7484-416b-9425-ca49f565e48e")]
    [ComVisible(true)]
    [SwAddin(Description = "test description", Title = "test", LoadAtStartup = true)]
    public class SwAddin : SolidWorks.Interop.swpublished.SwAddin
    {
        #region "Local Variables"
        SldWorks iSwApp;
        ICommandManager iCmdMgr;
        int addinID;
        Hashtable openDocs;
        SldWorks SwEventPtr;
        UserPMPPage ppage;
        // Public Properties
        public SldWorks SwApp
        {
            get { return iSwApp; }
        }
        public ICommandManager CmdMgr
        {
            get { return iCmdMgr; }
        }
        public Hashtable OpenDocumentsTable
        {
            get { return openDocs; }
        }
        #endregion
        #region "SOLIDWORKS Registration"
        [ComRegisterFunction()]
        public static void RegisterFunction(Type t)
        {
            // Get Custom Attribute: SwAddinAttribute
            object[] attributes = null;
            SwAddinAttribute SwAttr = null;
        }
    }
}
```

```

        attributes = System.Attribute.GetCustomAttributes(typeof(SwAddin), typeof(SwAddinAttribute));
        if (attributes.Length > 0)
        {
            SwAttr = (SwAddinAttribute)attributes[0];
        }
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;
        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "\\";
        Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
        addinkey.SetValue(null, 0);
        addinkey.SetValue("Description", SwAttr.Description);
        addinkey.SetValue("Title", SwAttr.Title);
        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "\\";
        addinkey = hkcu.CreateSubKey(keyname);
        addinkey.SetValue(null, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord);
    }
    [ComUnregisterFunction()]
    public static void UnregisterFunction(Type t)
    {
        Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
        Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;
        string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "\\";
        hklm.DeleteSubKey(keyname);
        keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "\\";
        hkcu.DeleteSubKey(keyname);
    }
}
#endregion
#region "ISwAddin Implementation"
public bool ConnectToSW(object ThisSW, int Cookie)
{
    iSwApp = (SldWorks)ThisSW;
    addinID = Cookie;
    // Setup callbacks
    iSwApp.SetAddinCallbackInfo(0, this, addinID);
    // Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(Cookie);
    AddCommandMgr();
    //Setup the Event Handlers
    SwEventPtr = iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    //Setup Sample Property Manager
    AddPMP();
    return true;
}
public bool DisconnectFromSW()
{
    RemoveCommandMgr();
    RemovePMP();
    DetachEventHandlers();
    iSwApp = null;
    //The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect();
    return true;
}
}
#endregion
#region "UI Methods"
public void AddCommandMgr()
{
    ICommandGroup cmdGroup = default(ICommandGroup);
    BitmapHandler iBmp = new BitmapHandler();
    Assembly thisAssembly = default(Assembly);

```

```

        int cmdIndex0 = 0;
        int cmdIndex1 = 0;
        string Title = "C# Addin";
        string ToolTip = "C# Addin";
        int[] docTypes = { (int)swDocumentTypes_e.swDocASSEMBLY, (int)swDocumentTypes_e.swDocDRAWING,
(int)swDocumentTypes_e.swDocPART };
        thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());
        cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1);
        cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("test.ToolbarLarge.bmp", thisAssembly);
        cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("test.ToolbarSmall.bmp", thisAssembly);
        cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("test.MainIconLarge.bmp", thisAssembly);
        cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("test.MainIconSmall.bmp", thisAssembly);
        cmdIndex0 = cmdGroup.AddCommandItem2("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0, 1);
        cmdIndex1 = cmdGroup.AddCommandItem2("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP",
"PMPEnable", 2, 1);
        cmdGroup.HasToolBar = true;
        cmdGroup.HasMenu = true;
        cmdGroup.Activate();
        foreach (int docType in docTypes)
        {
            ICommandTab cmdTab = iCmdMgr.GetCommandTab(docType, Title);
            bool bResult = false;
            if (cmdTab == null)
            {
                cmdTab = iCmdMgr.AddCommandTab(docType, Title);
                CommandTabBox cmdBox = cmdTab.AddCommandTabBox();
                int[] cmdIDs = new int[4];
                int[] TextType = new int[4];
                cmdIDs[0] = cmdGroup.get_CommandID(cmdIndex0);
                TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;
                cmdIDs[1] = cmdGroup.get_CommandID(cmdIndex1);
                TextType[1] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;
                cmdIDs[2] = cmdGroup.ToolbarId;
                TextType[2] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;
                bResult = cmdBox.AddCommands(cmdIDs, TextType);
                // Call GetCommands to confirm number of commands added
                int buttonNumber = 0;
                object idObject = null;
                object textTypeObject = null;
                buttonNumber = cmdBox.GetCommands(out idObject, out textTypeObject);
                Debug.Print("Added " + buttonNumber + " commands.");
                CommandTabBox cmdBox1 = cmdTab.AddCommandTabBox();
                cmdIDs = new int[1];
                TextType = new int[1];
                cmdIDs[0] = cmdGroup.ToolbarId;
                TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow;
                bResult = cmdBox1.AddCommands(cmdIDs, TextType);
                cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId);

            }
        }

        thisAssembly = null;
        iBmp.Dispose();
    }
    public void RemoveCommandMgr()
    {
        iCmdMgr.RemoveCommandGroup(1);
    }
    public bool AddPMP()
    {
        ppage = new UserPMPPage(this);
        return true;
    }

```

```

}
public bool RemovePMP()
{
    ppage = null;
    return true;
}
#endregion
#region "Event Methods"
public void AttachEventHandlers()
{
    AttachSWEvents();
    //Listen for events on all currently open docs
    AttachEventsToAllDocuments();
}
public void DetachEventHandlers()
{
    DetachSWEvents();
    //Close events on all currently open docs
    DocumentEventHandler docHandler = default(DocumentEventHandler);
    IDictionaryEnumerator _enumerator = openDocs.GetEnumerator();
    while (_enumerator.MoveNext())
    {
        docHandler = (DocumentEventHandler)_enumerator.Value;
        docHandler.DetachEventHandlers();
        //This also removes the pair from the hash
        docHandler = null;
        //key = null;
    }
}
public void AttachSWEvents()
{
    iSwApp.ActiveDocChangeNotify += this.SldWorks_ActiveDocChangeNotify;
    iSwApp.DocumentLoadNotify2 += this.SldWorks_DocumentLoadNotify2;
    iSwApp.FileNewNotify2 += this.SldWorks_FileNewNotify2;
    iSwApp.ActiveModelDocChangeNotify += this.SldWorks_ActiveModelDocChangeNotify;
    iSwApp.FileOpenPostNotify += this.SldWorks_FileOpenPostNotify;
}
public void DetachSWEvents()
{
    iSwApp.ActiveDocChangeNotify -= this.SldWorks_ActiveDocChangeNotify;
    iSwApp.DocumentLoadNotify2 -= this.SldWorks_DocumentLoadNotify2;
    iSwApp.FileNewNotify2 -= this.SldWorks_FileNewNotify2;
    iSwApp.ActiveModelDocChangeNotify -= this.SldWorks_ActiveModelDocChangeNotify;
    iSwApp.FileOpenPostNotify -= this.SldWorks_FileOpenPostNotify;
}
public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = default(ModelDoc2);
    modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while ((modDoc != null))
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}
public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
    {

```

```

        return false;
    }
    DocumentEventHandler docHandler = null;
    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
                docHandler = new PartEventHandler(modDoc, this);
                break;
            case (int)swDocumentTypes_e.swDocASSEMBLY:
                docHandler = new AssemblyEventHandler(modDoc, this);
                break;
            case (int)swDocumentTypes_e.swDocDRAWING:
                docHandler = new DrawingEventHandler(modDoc, this);
                break;
        }

        docHandler.AttachEventHandlers();
        openDocs.Add(modDoc, docHandler);
    }
    return true;
}

public void DetachModelEventHandler(ModelDoc2 modDoc)
{
    openDocs.Remove(modDoc);
    modDoc = null;
}

#endregion
#region "Event Handlers"
public int SldWorks_ActiveDocChangeNotify()
{
    //TODO: Add your implementation here
    return 0;
}

public int SldWorks_DocumentLoadNotify2(string docTitle, string docPath)
{
    ModelDoc2 modDoc = default(ModelDoc2);
    modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while ((modDoc != null))
    {
        if (modDoc.GetTitle() == docTitle)
        {
            if (!openDocs.Contains(modDoc))
            {
                AttachModelDocEventHandler(modDoc);
            }
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
    return 0;
}

public int SldWorks_FileNewNotify2(object newDoc, int doctype, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int SldWorks_ActiveModelDocChangeNotify()
{
    //TODO: Add your implementation here

```

```

        return 0;
    }
    public int SldWorks_FileOpenPostNotify(string FileName)
    {
        AttachEventsToAllDocuments();
        return 0;
    }
    #endregion
    #region "UI Callbacks"
    public void CreateCube()
    {
        //make sure we have a part open
        string partTemplate = null;
        ModelDoc2 model = default(ModelDoc2);
        FeatureManager featMan = default(FeatureManager);
        partTemplate = iSwApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
        model = (ModelDoc2)iSwApp.NewDocument(partTemplate, (int)swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0);
        model.InsertSketch2(true);
        model.SketchRectangle(0, 0, 0, 0.1, 0.1, 0.1, false);
        //Extrude the sketch
        featMan = model.FeatureManager;
        featMan.FeatureExtrusion(true, false, false, (int)swEndConditions_e.swEndCondBlind,
        (int)swEndConditions_e.swEndCondBlind, 0.1, 0.0, false, false, false,
        false, 0.0, 0.0, false, false, false, false, true, false, false
        );
    }
    public void ShowPMP()
    {
        ppage.Show();
    }
    public int PMPEnable()
    {
        int functionReturnValue = 0;
        if (iSwApp.ActiveDoc == null)
        {
            functionReturnValue = 0;
        }
        else
        {
            functionReturnValue = 1;
        }
        return functionReturnValue;
    }
    #endregion
}
}

```

Create CommandManager Tab and Tab Boxes Example (VB.NET)

This example shows how to create an add-in that creates a CommandManager tab and tab boxes.

NOTES:

- This sample code is part of a project that was built using the VB.NET Add-In Wizard.
- To run this sample code, you need to create a complete VB.NET project.
 1. Copy this sample code into **SwAddin.vb** of the new project.
 2. Add references as needed to debug and run the program.
- Your add-in must check to see if the tab already exists. If the tab already exists, then you should use that tab instead of creating a new tab. If your add-in does not check for an existing tab, then SOLIDWORKS Design will create a new tab each time it starts up.
- Users can add buttons to and remove buttons from your CommandManager tab. However, if your add-in removes the CommandManager tab upon exiting SOLIDWORKS Design, then any user customizations will be lost.

'-----

' SwAddin.vb

```
Imports System
Imports System.Windows.Forms
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File

<Guid("c5ede50f-7484-416b-9425-ca49f565e48e")> _
<ComVisible(True)> _
<SwAddin( _
    Description:="test description", _
    Title:="test", _
    LoadAtStartup:=True _
)> _
Public Class SwAddin
Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
Dim WithEvents iSwApp As SldWorks
Dim iCmdMgr As ICommandManager
Dim addinID As Integer
Dim openDocs As Hashtable
Dim SwEventPtr As SldWorks
Dim ppage As UserPMPage
' Public Properties
ReadOnly Property SwApp() As SldWorks
    Get
        Return iSwApp
    End Get
End Property
ReadOnly Property CmdMgr() As ICommandManager
    Get
        Return iCmdMgr
    End Get
End Property
ReadOnly Property OpenDocumentsTable() As Hashtable
    Get
        Return openDocs
    End Get
End Property
#End Region

#Region "SOLIDWORKS Registration"
<ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)
    ' Get Custom Attribute: SwAddinAttribute
    Dim attributes() As Object
    Dim SWattr As SwAddinAttribute = Nothing
    attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))
    If attributes.Length > 0 Then
        SWattr = DirectCast(attributes(0), SwAddinAttribute)
    End If
End Sub
End Class
```

```

Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser
Dim keyname As String = "SOFTWARE\SOLIDWORKS\AddIns\{" + t.GUID.ToString() + "}"
Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
addinkey.SetValue(Nothing, 0)
addinkey.SetValue("Description", SWattr.Description)
addinkey.SetValue("Title", SWattr.Title)
keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
addinkey = hkcu.CreateSubKey(keyname)
addinkey.SetValue(Nothing, SWattr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
    Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser
    Dim keyname As String = "SOFTWARE\SOLIDWORKS\AddIns\{" + t.GUID.ToString() + "}"
    hklm.DeleteSubKey(keyname)
    keyname = "Software\SOLIDWORKS\AddInsStartup\{" + t.GUID.ToString() + "}"
    hkcu.DeleteSubKey(keyname)
End Sub

#End Region

#Region "ISwAddin Implementation"
Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.ConnectToSW
    iSwApp = ThisSW
    addinID = Cookie
    ' Setup callbacks
    iSwApp.SetAddinCallbackInfo(0, Me, addinID)
    ' Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(Cookie)
    AddCommandMgr()
    'Setup the Event Handlers
    SwEventPtr = iSwApp
    openDocs = New Hashtable
    AttachEventHandlers()
    'Setup Sample Property Manager
    AddPMP()
    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    RemoveCommandMgr()
    RemovePMP()
    DetachEventHandlers()
    iSwApp = Nothing
    'The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect()
    DisconnectFromSW = True
End Function

#End Region

#Region "UI Methods"
Public Sub AddCommandMgr()
    Dim cmdGroup As ICommandGroup
    Dim iBmp As New BitmapHandler
    Dim thisAssembly As Assembly

    Dim cmdIndex0 As Integer, cmdIndex1 As Integer
    Dim Title As String = "VB Addin"
    Dim ToolTip As String = "VB Addin"

    Dim docTypes() As Integer = {swDocumentTypes_e.swDocASSEMBLY, _
        swDocumentTypes_e.swDocDRAWING, _
        swDocumentTypes_e.swDocPART}

    thisAssembly = System.Reflection.Assembly.GetAssembly(Me.GetType())
    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1)
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("test.ToolbarLarge.bmp", thisAssembly)
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("test.ToolbarSmall.bmp", thisAssembly)
    cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("test.MainIconLarge.bmp", thisAssembly)
    cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("test.MainIconSmall.bmp", thisAssembly)
    cmdIndex0 = cmdGroup.AddCommandItem2("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0, 1)
    cmdIndex1 = cmdGroup.AddCommandItem2("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP", "PMPEnable", 2, 1)
    cmdGroup.HasToolbar = True
    cmdGroup.HasMenu = True
    cmdGroup.Activate()
    For Each docType As Integer In docTypes
        Dim cmdTab As ICommandTab = iCmdMgr.GetCommandTab(docType, Title)
        Dim bResult As Boolean
    
```

```

If cmdTab Is Nothing Then
    cmdTab = iCmdMgr.AddCommandTab(docType, Title)
    Dim cmdBox As CommandTabBox = cmdTab.AddCommandTabBox
    Dim cmdIDs(3) As Integer
    Dim TextType(3) As Integer
    cmdIDs(0) = cmdGroup.CommandID(cmdIndex0)
    TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal
    cmdIDs(1) = cmdGroup.CommandID(cmdIndex1)
    TextType(1) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal
    cmdIDs(2) = cmdGroup.Toolbard
    TextType(2) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal
    bResult = cmdBox.AddCommands(cmdIDs, TextType)
    ' Call GetCommands to confirm
    Dim buttonNumber As Integer
    Dim idObject As Object = Nothing
    Dim textTypeObject As Object = Nothing
    buttonNumber = cmdBox.GetCommands(idObject, textTypeObject)
    Debug.Print("Added " + buttonNumber.ToString() + " commands.")
    Dim cmdBox1 As CommandTabBox = cmdTab.AddCommandTabBox()
    ReDim cmdIDs(1)
    ReDim TextType(1)
    cmdIDs(0) = cmdGroup.Toolbard
    TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow
    bResult = cmdBox1.AddCommands(cmdIDs, TextType)
    cmdTab.AddSeparator(cmdBox1, cmdGroup.Toolbard)
End If
Next
thisAssembly = Nothing
iBmp.Dispose()
End Sub
Public Sub RemoveCommandMgr()
    iCmdMgr.RemoveCommandGroup(1)
End Sub
Function AddPMP() As Boolean
    ppage = New UserPMPPage
    ppage.Init(iSwApp, Me)
End Function
Function RemovePMP() As Boolean
    ppage = Nothing
End Function
#End Region
#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()
    'Listen for events on all currently open docs
    AttachEventsToAllDocuments()
End Sub
Sub DetachEventHandlers()
    DetachSWEvents()
    'Close events on all currently open docs
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}
        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            docHandler.DetachEventHandlers() 'This also removes the pair from the hash
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub
Sub AttachSWEvents()
    AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify

```

```

End Sub

Sub DetachSWEvents()
    RemoveHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
    RemoveHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
    RemoveHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
    RemoveHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
    RemoveHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing
    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select
        docHandler.Init(iSwApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub

#End Region
#Region "Event Handlers"

Function SldWorks_ActiveDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
    Dim modDoc As ModelDoc2
    modDoc = iSwApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If modDoc.GetTitle = docTitle Then
            If Not openDocs.Contains(modDoc) Then
                AttachModelDocEventHandler(modDoc)
            End If
        End If
        modDoc = modDoc.GetNext()
    End While
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function

#End Region
#Region "UI Callbacks"

```

```

Sub CreateCube()
'make sure we have a part open
Dim partTemplate As String
Dim model As ModelDoc2
Dim featMan As FeatureManager
partTemplate = iSwApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swDefaultTemplatePart)
model = iSwApp.NewDocument(partTemplate, swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0)
model.InsertSketch2(True)
model.SketchRectangle(0, 0, 0, 0.1, 0.1, 0.1, False)
'Extrude the sketch
featMan = model.FeatureManager
featMan.FeatureExtrusion(True, _
    False, False, _
    swEndConditions_e.swEndCondBlind, swEndConditions_e.swEndCondBlind, _
    0.1, 0.0, _
    False, False, _
    False, False, _
    0.0, 0.0, _
    False, False, _
    False, False, _
    True, _
    False, False)
End Sub

Sub ShowPMP()
    ppage.Show()
End Sub

Function PMPEnable() As Integer
    If iSwApp.ActiveDoc Is Nothing Then
        PMPEnable = 0
    Else
        PMPEnable = 1
    End If
End Function

#End Region
End Class

```

Create CommandManager Tab Example (VB.NET)

This example shows how to create a CommandManager command tab.

NOTES:

- This example is only a code snippet. The complete project is available in the SOLIDWORKS API SDK.
- Your add-in must check to see if the tab already exists. If the tab already exists, then you must use that tab instead of creating a new tab. If your add-in does not check for an existing tab, then SOLIDWORKS Design will create a new tab each time it starts up.

```

...
Dim cmdGroup As ICommandGroup
Dim iBmp As New BitmapHandler
Dim thisAssembly As Assembly

Dim cmdIndex0 As Integer, cmdIndex1 As Integer
Dim Title As String = "VB Addin"
Dim ToolTip As String = "VB Addin"

Dim docTypes() As Integer = {swDocumentTypes_e.swDocASSEMBLY, _
    swDocumentTypes_e.swDocDRAWING, _
    swDocumentTypes_e.swDocPART}

thisAssembly = System.Reflection.Assembly.GetAssembly(Me.GetType())

cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1)
cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin3.ToolbarLarge.bmp", thisAssembly)
cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("SwVBAddin3.ToolbarSmall.bmp", thisAssembly)
cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("SwVBAddin3.MainIconLarge.bmp", thisAssembly)
cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("SwVBAddin3.MainIconSmall.bmp", thisAssembly)

cmdIndex0 = cmdGroup.AddCommandItem("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0)
cmdIndex1 = cmdGroup.AddCommandItem("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP", "PMPEnable", 2)

cmdGroup.HasToolBar = True
cmdGroup.HasMenu = True
cmdGroup.Activate()

For Each docType As Integer In docTypes
    Dim cmdTabs() As Object = iCmdMgr.CommandTabs(docType)
    Dim cmdTab As ICommandTab

    Dim tabExists As Boolean = False

    For i As Integer = 0 To iCmdMgr.GetCommandTabCount(docType) - 1
        cmdTab = cmdTabs(i)
        If cmdTab.TabName = Title Then
            tabExists = True
            Exit For
        End If
    Next

    If Not tabExists Then
        cmdTab = iCmdMgr.CreateCommandTab(-1, Title, docType)

        cmdTab.AddCommand(cmdGroup.CommandID(cmdIndex0))
        cmdTab.SetTextDisplay(cmdGroup.CommandID(cmdIndex0), swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal)

        cmdTab.AddCommand(cmdGroup.CommandID(cmdIndex1))
        cmdTab.SetTextDisplay(cmdGroup.CommandID(cmdIndex1), swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal)

        cmdTab.AddCommand(cmdGroup.ToolbarId)
        cmdTab.SetTextDisplay(cmdGroup.ToolbarId, swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow)

        cmdTab.AddSeparator(cmdGroup.ToolbarId)
    End If
Next

```

```
thisAssembly = Nothing
```

```
iBmp.Dispose()
```

```
End Sub
```

```
...
```

Create Compound Note Example (VBA)

This example shows how to create a compound note in a drawing document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.
' 2. Click the letter E in the drawing view.
'
' Postconditions:
' 1. Creates a compound note.
' 2. Click the compound note and drag it to a different
'    location in the drawing.
'
' NOTE: Because this drawing is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swNote As SldWorks.note

Sub main()

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swDrawing = swModel

    ' Create the compound note
    makeCnote

End Sub

Sub makeCnote()

    Set swNote = swDrawing.CreateCompoundNote(0.00635, 0.05, 0.2, 0#)

    ' Add text to note
    swNote.AddText "zero pt", 0, 0, 0
    swNote.AddText "top", 0.01, 0.01, 0

    ' Prepare note for geometry
    swNote.BeginSketchEdit

    ' Add geometry to note
    swDrawing.CreateLineVB 0, 0, 0, 0.01, 0.01, 0
    swDrawing.CreateCircleVB 0, 0, 0, 0.01414

    ' Finish
    swNote.EndSketchEdit

    ' Do some queries
    If (swNote.IsCompoundNote()) Then
        Dim count As Long
        Dim n2 As String
        Dim n1 As String
        Dim ht2 As Double
        ' If object is compound note
        ' Get number of text items
        count = swNote.GetTextCount()
        ' Get the second piece of text
        n2 = swNote.GetTextAtIndex(2)
        ' Get the first piece of text
        n1 = swNote.GetTextAtIndex(1)
        ' Get height of 2nd piece of text
        ht2 = swNote.GetHeightAtIndex(2)

    End If

End Sub
```

Create ConvertSolid Feature Example (C#)

This example shows how to convert a solid body to sheet metal.

// -----

// Preconditions:

// 1. Open *public_documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\api\sweepcutextrude.sldprt*.

// 2. Ensure that the Sheet Metal tab is visible on the Command Manager toolbar.

// 3. On the SOLIDWORKS toolbar, click **Tools > Macro > New** and create a C# macro.

// 4. Copy and paste the code below into **SolidWorksMacro.cs**.

// 5. Open an Immediate window.

//

// Postconditions:

// 1. Creates **Convert-Solid1** and **Sheet-Metal1**.

// 2. Inspect the FeatureManager design tree and the Immediate window.

//

// NOTE: Because the model may be used elsewhere, do not save changes to it.

// -----

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace ConvertToSheetMetal_CSharp
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            Feature swFeat;
            FeatureManager FeatMgr;
            ConvertSolidFeatureData swConvertSolidFeatData;
            ConvertSolidFeatureData FeatData;
            bool boolstatus;
            CustomBendAllowance cba;
            Face2 face;
            object faceId;
            object bendEdges;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            FeatMgr = swModel.FeatureManager;

            boolstatus = swModel.Extension.SelectByID("", "FACE", 0.0004130570195002, 0.02357994168921, 0.02568415695742, true, 0,
            null, 0);
            boolstatus = swModel.Extension.SelectByID("", "EDGE", -0.00190522473838, 0.02387533864419, 0.04979931166838, true, 1,
            null, 0);
            boolstatus = swModel.Extension.SelectByID("", "EDGE", 0.02911271681069, 0.02376277320678, 0.02892436699148, true, 1,
            null, 0);
```

```

    boolstatus = swModel.Extension.SelectByID2("", "EDGE", -0.004838857104858, 0.02387396382323, -0.0001997542986487, true,
1, null, 0);

    // Create a feature data object
    swConvertSolidFeatData =
(ConvertSolidFeatureData)FeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmSolidToSheetMetal);

    cba = (CustomBendAllowance)swConvertSolidFeatData.GetCustomBendAllowance();

    cba.Type = (int)swBendAllowanceTypes_e.swBendAllowanceKFactor;
    cba.KFactor = 0.08;

    // Initialize the feature data object with custom bend allowance
    swConvertSolidFeatData.Initialize(true, false, cba);

    // Create the feature
    swFeat = FeatMgr.CreateFeature(swConvertSolidFeatData);

    FeatData = (ConvertSolidFeatureData)swFeat.GetDefinition();

    cba = (CustomBendAllowance)swConvertSolidFeatData.GetCustomBendAllowance();

    Debug.Print("Type of custom bend allowance as defined in swBendAllowanceTypes_e: " + cba.Type);
    Debug.Print("K factor: " + cba.KFactor);

    Debug.Print("SheetMetal thickness : " + FeatData.SheetThickness);
    Debug.Print("Bend Radius : " + FeatData.BendRadius);
    Debug.Print("Reverse thickness : " + FeatData.ReverseThickness);
    Debug.Print("Keep body : " + FeatData.KeepBody);
    Debug.Print("Overlap type : " + FeatData.CornerDefaults);
    Debug.Print("Rip gaps : " + FeatData.RipGap);
    Debug.Print("Rip overlap ratio : " + FeatData.RipOverlapRatio);
    Debug.Print("Auto relief type : " + FeatData.ReliefType);
    Debug.Print("Auto relief ratio : " + FeatData.ReliefRatio);

    bendEdges = FeatData.BendEdges;
    Debug.Print("Number of bend edges: " + FeatData.GetBendEdgesCount());

    FeatData.AccessSelections(swModel, null);

    face = (Face2)FeatData.GetFixedFace();
    faceId = face.GetFaceId();
    Debug.Print("Fixed face ID: " + faceId);

    // Modify initial values
    FeatData.SheetThickness = 0.014;
    FeatData.BendRadius = 0.0006;

    // Modify the feature definition
    swFeat.ModifyDefinition(FeatData, swModel, null);

    Debug.Print("-----After Setting Values-----");

    Debug.Print("SheetMetal thickness : " + FeatData.SheetThickness);
    Debug.Print("Bend Radius : " + FeatData.BendRadius);
    Debug.Print("Reverse thickness : " + FeatData.ReverseThickness);
    Debug.Print("Keep body : " + FeatData.KeepBody);
    Debug.Print("Overlap type : " + FeatData.CornerDefaults);
    Debug.Print("Rip gaps : " + FeatData.RipGap);
    Debug.Print("Rip overlap ratio : " + FeatData.RipOverlapRatio);
    Debug.Print("Auto relief type : " + FeatData.ReliefType);
    Debug.Print("Auto relief ratio : " + FeatData.ReliefRatio);
}

```

```
        public SldWorks swApp;  
    }  
}
```

Create ConvertSolid Feature Example (VB.NET)

This example shows how to convert a solid body to sheet metal.

Preconditions:

1. Open *public_documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\api\sweepcutextrude.sldprt*.
2. Ensure that the Sheet Metal tab is visible on the Command Manager toolbar.
3. Open an Immediate window.

Postconditions:

1. Creates **Convert-Solid1** and **Sheet-Metal1**.
2. Inspect the FeatureManager design tree and the Immediate window.

NOTE: Because the model may be used elsewhere, do not save changes to it.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim FeatMgr As FeatureManager
        Dim swConvertSolidFeatData As ConvertSolidFeatureData
        Dim FeatData As ConvertSolidFeatureData
        Dim boolstatus As Boolean
        Dim cba As CustomBendAllowance
        Dim face As Face2
        Dim faceId As Object
        Dim bendEdges As Object
```

```
        swModel = swApp.ActiveDoc
```

```
        FeatMgr = swModel.FeatureManager
```

```
        boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.0004130570195002, 0.02357994168921, 0.02568415695742, True, 0,
Nothing, 0)
```

```
        boolstatus = swModel.Extension.SelectByID2("", "EDGE", -0.00190522473838, 0.02387533864419, 0.04979931166838, True, 1,
Nothing, 0)
```

```
        boolstatus = swModel.Extension.SelectByID2("", "EDGE", 0.02911271681069, 0.02376277320678, 0.02892436699148, True, 1,
Nothing, 0)
```

```
        boolstatus = swModel.Extension.SelectByID2("", "EDGE", -0.004838857104858, 0.02387396382323, -0.0001997542986487, True, 1,
Nothing, 0)
```

```
        'Create a feature data object
```

```
        swConvertSolidFeatData = FeatMgr.CreateDefinition(swFeatureNameID_e.swFmSolidToSheetMetal)
```

```
        cba = swConvertSolidFeatData.GetCustomBendAllowance()
```

```
        cba.Type = swBendAllowanceTypes_e.swBendAllowanceKFactor
```

```
        cba.KFactor = 0.08
```

```
        'Initialize the feature data object with custom bend allowance
```

```
        swConvertSolidFeatData.Initialize(True, False, cba)
```

```
        'Create the feature
```

```
        swFeat = FeatMgr.CreateFeature(swConvertSolidFeatData)
```

```
        FeatData = swFeat.GetDefinition()
```

```

cba = swConvertSolidFeatData.GetCustomBendAllowance()

Debug.Print("Type of custom bend allowance as defined in swBendAllowanceTypes_e: " & cba.Type)
Debug.Print("K factor: " & cba.KFactor)

Debug.Print("SheetMetal thickness : " & FeatData.SheetThickness)
Debug.Print("Bend Radius : " & FeatData.BendRadius)
Debug.Print("Reverse thickness : " & FeatData.ReverseThickness)
Debug.Print("Keep body : " & FeatData.KeepBody)
Debug.Print("Overlap type : " & FeatData.CornerDefaults)
Debug.Print("Rip gaps : " & FeatData.RipGap)
Debug.Print("Rip overlap ratio : " & FeatData.RipOverlapRatio)
Debug.Print("Auto relief type : " & FeatData.ReliefType)
Debug.Print("Auto relief ratio : " & FeatData.ReliefRatio)

bendEdges = FeatData.BendEdges
Debug.Print("Number of bend edges: " & FeatData.GetBendEdgesCount)

FeatData.AccessSelections(swModel, Nothing)

face = FeatData.GetFixedFace
faceId = face.GetFaceId()
Debug.Print("Fixed face ID: " & faceId)

'Modify initial values
FeatData.SheetThickness = 0.014
FeatData.BendRadius = 0.0006

'Modify the feature definition
swFeat.ModifyDefinition(FeatData, swModel, Nothing)

Debug.Print("-----After Setting Values-----")

Debug.Print("SheetMetal thickness : " & FeatData.SheetThickness)
Debug.Print("Bend Radius : " & FeatData.BendRadius)
Debug.Print("Reverse thickness : " & FeatData.ReverseThickness)
Debug.Print("Keep body : " & FeatData.KeepBody)
Debug.Print("Overlap type : " & FeatData.CornerDefaults)
Debug.Print("Rip gaps : " & FeatData.RipGap)
Debug.Print("Rip overlap ratio : " & FeatData.RipOverlapRatio)
Debug.Print("Auto relief type : " & FeatData.ReliefType)
Debug.Print("Auto relief ratio : " & FeatData.ReliefRatio)

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Coordinate System Feature (VBA)

This example creates a coordinate system feature. The example then accesses the feature and edits it.

```
' -----
```

```
' Precondition: Model document is open.
```

```
'
```

```
' Postconditions: Coordinate system feature is created.
```

```
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel As SldWorks.ModelDoc2
```

```
Dim swModelDocExt As SldWorks.ModelDocExtension
```

```
Dim swSelMgr As SldWorks.SelectionMgr
```

```
Dim swFeat As SldWorks.Feature
```

```
Dim swFeatMgr As SldWorks.FeatureManager
```

```
Dim cSysData As SldWorks.CoordinateSystemFeatureData
```

```
Dim component As SldWorks.Component2
```

```
Dim boolstatus As Boolean
```

```
Dim vo As Variant
```

```
Dim vx As Variant
```

```
Dim vy As Variant
```

```
Dim vz As Variant
```

```
Dim evo As Variant
```

```
Dim evx As Variant
```

```
Dim evy As Variant
```

```
Dim evz As Variant
```

```
Dim so(0) As Object
```

```
Dim sx(1) As Object
```

```
Dim sy(1) As Object
```

```
Dim sz(1) As Object
```

```
Dim eso(0) As Object
```

```
Dim esx(1) As Object
```

```
Dim esy(1) As Object
```

```
Dim esz(1) As Object
```

```
Dim xEnt As Object
```

```
Sub main()
```

```
Set swApp = Application.SldWorks
```

```
Set swModel = swApp.ActiveDoc
```

```
Set swSelMgr = swModel.SelectionManager
```

```
Set swModelDocExt = swModel.Extension
```

```
Set swFeatMgr = swModel.FeatureManager
```

```
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 0, 0.00635, 0, False, 0, Nothing, 0)
```

```
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.0308159564126, 0.006349999999827, 0.01220751949762, True, 0, Nothing, 0)
```

```
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 0.0762, 0.00635, 0.01905, True, 0, Nothing, 0)
```

```
boolstatus = swModelDocExt.SelectByID2("", "VERTEX", 0.0762, 0.00635, 0, True, 0, Nothing, 0)
```

```
' Assume a vertex was selected for the origin
```

```
Set so(0) = swSelMgr.GetSelectedObject6(1, -1)
```

```
' Assume one entity (point, line/edge, plane/face) was selected for the x axis
```

```
Set sx(0) = swSelMgr.GetSelectedObject6(2, -1)
```

```
' Assume two entities (point, line/edge, plane/face) were selected for the y axis
```

```
Set sy(0) = swSelMgr.GetSelectedObject6(3, -1)
```

```
Set sy(1) = swSelMgr.GetSelectedObject6(4, -1)
```

```
' Try getting additional selected entities for the z axis
```

```
Set sz(0) = swSelMgr.GetSelectedObject6(5, -1)
```

```
Set sz(1) = swSelMgr.GetSelectedObject6(6, -1)
```

```

vo = so
vx = sx
vy = sy
vz = sz

swModel.ClearSelection2 True

' Create coordinate system
Set swFeat = swFeatMgr.CreateCoordinateSystem(so(0), (vx), (vy), (vz))

If swFeat Is Nothing Then
    Debug.Print "Did not create a coordinate system feature!"
Else
    Set cSysData = swFeat.GetDefinition
    boolstatus = cSysData.AccessSelections(swModel, component)

    ' Get and output the transform
    Dim swXform As SldWorks.MathTransform
    Set swXform = cSysData.Transform
    Debug.Print "    Origin = (" & -1# * swXform.ArrayData(9) * 1000# & ", " & -1# * swXform.ArrayData(10) * 1000# & ", " & -1# * swXform.ArrayData(11) * 1000# & ") mm"
    Debug.Print "    Rot1  = (" & swXform.ArrayData(0) & ", " & swXform.ArrayData(1) & ", " & swXform.ArrayData(2) & ")"
    Debug.Print "    Rot2  = (" & swXform.ArrayData(3) & ", " & swXform.ArrayData(4) & ", " & swXform.ArrayData(5) & ")"
    Debug.Print "    Rot3  = (" & swXform.ArrayData(6) & ", " & swXform.ArrayData(7) & ", " & swXform.ArrayData(8) & ")"
    Debug.Print "    Trans = (" & swXform.ArrayData(9) * 1000# & ", " & swXform.ArrayData(10) * 1000# & ", " & swXform.ArrayData(11) * 1000# & ") mm"
    Debug.Print "    Scale = " & swXform.ArrayData(12)

    Debug.Print "XFlipped? " & cSysData.XFlipped
    cSysData.XFlipped = True

    ' Select new entities for editing the coordinate system feature
    boolstatus = swModelDocExt.SelectByID("", "VERTEX", 0, 0.00635, 0.01905, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("", "EDGE", 0.03505266269053, -6.420391980555E-05, 0.01914079805448, True, 2, Nothing, 0)
    ' Assume a vertex was selected for the origin
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.0308159564126, 0.006349999999827, 0.01220751949762, True, 3, Nothing, 0)

    ' Edit the origin
    Set eso(0) = swSelMgr.GetSelectedObject6(1, -1)

    ' Edit the y axis
    ' Assume two entities (point, line/edge, plane/face) were selected for the y axis

    Set esy(0) = swSelMgr.GetSelectedObject6(2, -1)
    Set esz(0) = swSelMgr.GetSelectedObject6(3, -1)

    evo = eso
    evy = esy
    evz = esz

swModel.ClearSelection2 True

cSysData.OriginEntity = eso(0)
cSysData.YAxisEntities = evy

' Try appending z-axis entities to see if they replace x-axis entity
' If this happens, then y and z are the active axes,
' and x-axis entity is inaccessible
cSysData.ZAxisEntities = evz

' Get the origin entity
Dim entType As Long

```

```

Stop
Dim geo As SldWorks.Entity
Set geo = cSysData.OriginEntity
entType = geo.GetType

' Get the x-axis entities; there shouldn't be any, because x is not active
Dim gv As Variant
Dim vCount As Long
vCount = cSysData.GetXAxisEntitiesCount
gv = cSysData.XAxisEntities

Dim i As Long

' Get the y-axis entities
vCount = cSysData.GetYAxisEntitiesCount
gv = cSysData.YAxisEntities

For i = 0 To UBound(gv)
    Set xEnt = gv(i)
    entType = xEnt.GetType
Next i

boolstatus = swFeat.ModifyDefinition(cSysData, swModel, Nothing)

cSysData.ReleaseSelectionAccess
End If

swModel.EditRebuild3

End Sub

```

Create Corner Relief Feature Example (C#)

This example shows how to create a corner relief feature.

```
//-----
// Preconditions:
// Open public_documents\samples\tutorial\sheetmetal\formtools\cover.sldprt.
//
// Postconditions:
// 1. The model is rotated to the back view.
// 2. An edge flange is created.
// 3. The model is rotated slightly about the x-axis.
// 4. A corner relief feature is created:
//    * A rectangular corner relief is added to one corner of the edge flange.
//    * An obround corner relief is added to another corner of the edge flange.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace TwoCorners_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            bool bValue = false;
            Edge swEdge = default(Edge);
            double dAngle = 0;
            double dLength = 0;
            Feature swFeature = default(Feature);
            Entity swEntity = default(Entity);
            Sketch swSketch = default(Sketch);
            object[] vSketchSegments = null;
            SketchLine swSketchLine = default(SketchLine);
            SketchPoint swStartPoint = default(SketchPoint);
            SketchPoint swEndPoint = default(SketchPoint);
            int nOptions = 0;
            double dSize = 0;
            double dFactor1 = 0;
            double dFactor2 = 0;
            Edge[] aFlangeEdges = new Edge[1];
            object vFlangeEdges = null;
            Sketch[] aSketchFeats = new Sketch[1];
            object vSketchFeats = null;

            // Get active document
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Flange parameters

            // Set the angle
            dAngle = (90.0 / 180.0) * 3.1415926535897;

            dLength = 0.01;

            // Rotate model so that IModelDocExtension::SelectByID2 coordinates can be found
            swModel.ShowNamedView2("Back", -1);
            swModel.ViewZoomtofit2();

            // Select edge for flange
            bValue = swModel.Extension.SelectByID2("", "EDGE", 0.0372105002552985, 0.052846642716446, -9.93711211094706E-06, false,
0, null, 0);

            // Get edge
            swEdge = (Edge)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);

            // Insert a sketch for an edge flange
            swFeature = (Feature)swModel.InsertSketchForEdgeFlange(swEdge, dAngle, false);

            // Select
            bValue = swFeature.Select2(false, 0);

            // Start sketch editing
            swModel.EditSketch();

            // Get the active sketch
            swSketch = (Sketch)swModel.SketchManager.ActiveSketch;

            // Add the edge to the sketch

            // Cast edge to entity
            swEntity = (Entity)swEdge;

            // Select edge
            bValue = swEntity.Select4(false, null);
```

```

// Use the edge in the sketch
bValue = swModel.SketchManager.SketchUseEdge(false);

// Get the created sketch line
vSketchSegments = (object[])swSketch.GetSketchSegments();

swSketchLine = (SketchLine)vSketchSegments[0];

// Get start and end point
swStartPoint = (SketchPoint)swSketchLine.GetStartPoint2();
swEndPoint = (SketchPoint)swSketchLine.GetEndPoint2();

// Create additional lines to define sketch
// Set parameters defining the sketch geometry
dSize = swEndPoint.X - swStartPoint.X;
dFactor1 = 0.1;
dFactor2 = 1.25;

swModel.SetAddToDB(true);
swModel.SetDisplayWhenAdded(false);

swModel.SketchManager.CreateLine(swStartPoint.X, swStartPoint.Y, 0.0, swStartPoint.X, swStartPoint.Y + dLength, 0.0);
swModel.SketchManager.CreateLine(swStartPoint.X, swStartPoint.Y + dLength, 0.0, swStartPoint.X + dFactor1 * dSize,
swStartPoint.Y + dFactor2 * dLength, 0.0);
swModel.SketchManager.CreateLine(swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0,
swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0);
swModel.SketchManager.CreateLine(swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0,
swEndPoint.X, swEndPoint.Y + dLength, 0.0);
swModel.SketchManager.CreateLine(swEndPoint.X, swEndPoint.Y, 0.0, swEndPoint.X, swEndPoint.Y + dLength, 0.0);

// Reset
swModel.SetDisplayWhenAdded(true);
swModel.SetAddToDB(false);

// Commit changes made to the sketch
swModel.SketchManager.InsertSketch(true);

// Set options
nOptions = (int)swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRadius +
(int)swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRelief;

aFlangeEdges[0] = swEdge;
aSketchFeats[0] = swSketch;

vFlangeEdges = aFlangeEdges;
vSketchFeats = aSketchFeats;

swFeature = swModel.FeatureManager.InsertSheetMetalEdgeFlange2((vFlangeEdges), (vSketchFeats), nOptions, dAngle, 0.0,
(int)swFlangePositionTypes_e.swFlangePositionTypeBendOutside, dLength, (int)swSheetMetalReliefTypes_e.swSheetMetalReliefNone, 0.0,
0.0,
0.0, (int)swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp, null);

// Rotate view so that IModelDocExtension::SelectByID2 coordinates can be found
ModelView myModelView = default(ModelView);
myModelView = (ModelView)swModel.ActiveView;
myModelView.RotateAboutCenter(45, 0.00911235438195936);

// Select the sheet metal body to which to apply a corner relief
bValue = swModel.Extension.SelectByID2("Edge-Flange1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
swModel.ClearSelection2(true);

// Specify two corners of the edge flange for which to create a corner relief

// Select faces that define the first corner
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0549242492243928, 0.053073918098565, 0.0242634000000049, true, 4,
null, 0);
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0276778697571744, 0.0530739180985651, -0.00104170971004399, true,
4, null, 0);
long myCorner = 0;
myCorner = swModel.FeatureManager.AddCornerReliefCorner();

// Specify the type of corner relief to apply to the first corner
bool myReliefType = false;
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, (int)swCornerReliefType_e.swCornerSquareRelief, 0.0001,
0.0007366, 0.00018415, false, false, false, true, false);
swModel.ClearSelection2(true);

// Select faces that define the second corner
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0276778697571744, 0.0530739180985651, -0.00104170971004399, true,
4, null, 0);
bValue = swModel.Extension.SelectByID2("", "FACE", 0.000431490289955978, 0.053073918098565, 0.0242634000000049, true,
4, null, 0);
myCorner = swModel.FeatureManager.AddCornerReliefCorner();

// Specify the type of corner relief to apply to the second corner
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, (int)swCornerReliefType_e.swCornerObroundRelief, 0.0001,
0.0029464, 0.0007366, false, false, false, false, false);

// Create the corner relief feature
Feature myFeature = default(Feature);
myFeature = swModel.FeatureManager.FinishCornerRelief();
}

```

```
    public SldWorks swApp;  
}  
  
}
```

Create Corner Relief Feature Example (VBA)

This example shows how to create a corner relief feature.

```

'-----
' Preconditions:
' Open public_documents\samples\tutorial\sheetmetal\formtools\cover.sldprt.
'
' Postconditions:
' 1. The model is rotated to the back view.
' 2. An edge flange is created.
' 3. The model is rotated slightly about the x-axis.
' 4. A corner relief feature is created:
'     * A rectangular corner relief is added to one corner of the edge flange.
'     * An obround corner relief is added to another corner of the edge flange.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim bValue           As Boolean
    Dim swEdge           As SldWorks.Edge
    Dim dAngle           As Double
    Dim dLength          As Double
    Dim swFeature         As SldWorks.Feature
    Dim swEntity         As SldWorks.Entity
    Dim swSketch         As SldWorks.Sketch
    Dim vSketchSegments  As Variant
    Dim swSketchLine     As SldWorks.SketchLine
    Dim swStartPoint     As SldWorks.SketchPoint
    Dim swEndPoint       As SldWorks.SketchPoint
    Dim nOptions         As SwConst.swInsertEdgeFlangeOptions_e
    Dim dSize            As Double
    Dim dFactor1         As Double
    Dim dFactor2         As Double
    Dim aFlangeEdges(0) As SldWorks.Edge
    Dim vFlangeEdges     As Variant
    Dim aSketchFeats(0) As SldWorks.Sketch
    Dim vSketchFeats     As Variant

    ' Connect to SOLIDWORKS
    Set swApp = Application.SldWorks

    ' Get active document
    Set swModel = swApp.ActiveDoc

    ' Flange parameters

    ' Set the angle
    dAngle = (90# / 180#) * 3.1415926535897

    dLength = 0.01

    ' Rotate model so that IModelDocExtension::SelectByID2 coordinates can be found
    swModel.ShowNamedView2 "Back", -1
    swModel.ViewZoomtofit2

    ' Select edge for flange
    bValue = swModel.Extension.SelectByID2("", "EDGE", 3.72105002552985E-02, 0.052846642716446, -9.93711211094706E-06, False, 0,
    Nothing, 0)

    ' Get edge
    Set swEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Insert a sketch for an edge flange
    Set swFeature = swModel.InsertSketchForEdgeFlange(swEdge, dAngle, False)

    ' Select
    bValue = swFeature.Select2(False, 0)

    ' Start sketch editing
    swModel.EditSketch

    ' Get the active sketch
    Set swSketch = swModel.SketchManager.ActiveSketch

    ' Add the edge to the sketch

    ' Cast edge to entity
    Set swEntity = swEdge

    ' Select edge
    bValue = swEntity.Select4(False, Nothing)

    ' Use the edge in the sketch
    bValue = swModel.SketchManager.SketchUseEdge(False)

    ' Get the created sketch line
    vSketchSegments = swSketch.GetSketchSegments

    Set swSketchLine = vSketchSegments(0)

    ' Get start and end point
    Set swStartPoint = swSketchLine.GetStartPoint2
    Set swEndPoint = swSketchLine.GetEndPoint2

```

```

' Create additional lines to define sketch
' Set parameters defining the sketch geometry
dSize = swEndPoint.X - swStartPoint.X
dFactor1 = 0.1
dFactor2 = 1.25

swModel.SetAddToDB True
swModel.SetDisplayWhenAdded False

swModel.SketchManager.CreateLine swStartPoint.X, swStartPoint.Y, 0#, swStartPoint.X, swStartPoint.Y + dLength, 0#
swModel.SketchManager.CreateLine swStartPoint.X, swStartPoint.Y + dLength, 0#, swStartPoint.X + dFactor1 * dSize,
swStartPoint.Y + dFactor2 * dLength, 0#
swModel.SketchManager.CreateLine swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X -
dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#
swModel.SketchManager.CreateLine swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0#, swEndPoint.X,
swEndPoint.Y + dLength, 0#
swModel.SketchManager.CreateLine swEndPoint.X, swEndPoint.Y, 0#, swEndPoint.X, swEndPoint.Y + dLength, 0#

' Reset
swModel.SetDisplayWhenAdded True
swModel.SetAddToDB False

' Commit changes made to the sketch
swModel.SketchManager.InsertSketch True

' Set options
nOptions = swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRadius +
swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRelief

Set aFlangeEdges(0) = swEdge
Set aSketchFeats(0) = swSketch

vFlangeEdges = aFlangeEdges
vSketchFeats = aSketchFeats

Set swFeature = swModel.FeatureManager.InsertSheetMetalEdgeFlange2((vFlangeEdges), (vSketchFeats), nOptions, dAngle, 0#,
swFlangePositionTypes_e.swFlangePositionTypeBendOutside, dLength, swSheetMetalReliefTypes_e.swSheetMetalReliefNone, 0#, 0#, 0#,
swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp, Nothing)

' Rotate view so that IModelDocExtension::SelectByID2 coordinates can be found
Dim myModelView As SldWorks.ModelView
Set myModelView = swModel.ActiveView
myModelView.RotateAboutCenter 45, 9.11235438195936E-03

' Select the sheet metal body to which to apply a corner relief
bValue = swModel.Extension.SelectByID2("Edge-Flange1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
swModel.ClearSelection2 True

' Specify two corners of the edge flange for which to create a corner relief

' Select faces that define the first corner
bValue = swModel.Extension.SelectByID2("", "FACE", 5.49242492243928E-02, 0.053073918098565, 2.42634000000049E-02, True, 4,
Nothing, 0)
bValue = swModel.Extension.SelectByID2("", "FACE", 2.76778697571744E-02, 5.30739180985651E-02, -1.04170971004399E-03, True, 4,
Nothing, 0)
Dim myCorner As Long
myCorner = swModel.FeatureManager.AddCornerReliefCorner()

' Specify the type of corner relief to apply to the first corner
Dim myReliefType As Boolean
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, swCornerSquareRelief, 0.0001, 0.0007366, 0.00018415, False,
False, False, True, False)
swModel.ClearSelection2 True

' Select faces that define the second corner
bValue = swModel.Extension.SelectByID2("", "FACE", 2.76778697571744E-02, 5.30739180985651E-02, -1.04170971004399E-03, True, 4,
Nothing, 0)
bValue = swModel.Extension.SelectByID2("", "FACE", 4.31490289955978E-04, 0.053073918098565, 2.42634000000049E-02, True, 4,
Nothing, 0)
myCorner = swModel.FeatureManager.AddCornerReliefCorner()

' Specify the type of corner relief to apply to the second corner
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, swCornerOroundRelief, 0.0001, 0.0029464, 0.0007366, False,
False, False, False, False)

' Create the corner relief feature
Dim myFeature As SldWorks.Feature
Set myFeature = swModel.FeatureManager.FinishCornerRelief()

```

End Sub

Create Corner Relief Feature Example (VB.NET)

This example shows how to create a corner relief feature.

```

'-----
' Preconditions:
' Open public_documents\samples\tutorial\sheetmetal\formtools\cover.sldprt.
'-----
' Postconditions:
' 1. The model is rotated to the back view.
' 2. An edge flange is created.
' 3. The model is rotated slightly about the x-axis.
' 4. A corner relief feature is created:
'    * A rectangular corner relief is added to one corner of the edge flange.
'    * An obround corner relief is added to another corner of the edge flange.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim bValue As Boolean
        Dim swEdge As Edge
        Dim dAngle As Double
        Dim dLength As Double
        Dim swFeature As Feature
        Dim swEntity As Entity
        Dim swSketch As Sketch
        Dim vSketchSegments As Object
        Dim swSketchLine As SketchLine
        Dim swStartPoint As SketchPoint
        Dim swEndPoint As SketchPoint
        Dim nOptions As swInsertEdgeFlangeOptions_e
        Dim dSize As Double
        Dim dFactor1 As Double
        Dim dFactor2 As Double
        Dim aFlangeEdges(0) As Edge
        Dim vFlangeEdges As Object
        Dim aSketchFeats(0) As Sketch
        Dim vSketchFeats As Object

        ' Get active document
        swModel = swApp.ActiveDoc

        ' Flange parameters

        ' Set the angle
        dAngle = (90.0# / 180.0#) * 3.1415926535897

        dLength = 0.01

        ' Rotate model so that IModelDocExtension::SelectByID2 coordinates can be found
        swModel.ShowNamedView2("Back", -1)
        swModel.ViewZoomtofit2()

        ' Select edge for flange
        bValue = swModel.Extension.SelectByID2("", "EDGE", 0.0372105002552985, 0.052846642716446, -0.00000993711211094706, False,
0, Nothing, 0)

        ' Get edge
        swEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)

        ' Insert a sketch for an edge flange
        swFeature = swModel.InsertSketchForEdgeFlange(swEdge, dAngle, False)

        ' Select
        bValue = swFeature.Select2(False, 0)

        ' Start sketch editing
        swModel.EditSketch()

        ' Get the active sketch
        swSketch = swModel.SketchManager.ActiveSketch

        ' Add the edge to the sketch

        ' Cast edge to entity
        swEntity = swEdge

        ' Select edge
        bValue = swEntity.Select4(False, Nothing)

        ' Use the edge in the sketch
        bValue = swModel.SketchManager.SketchUseEdge(False)
    
```

```

' Get the created sketch line
vSketchSegments = swSketch.GetSketchSegments

swSketchLine = vSketchSegments(0)

' Get start and end point
swStartPoint = swSketchLine.GetStartPoint2
swEndPoint = swSketchLine.GetEndPoint2

' Create additional lines to define sketch
' Set parameters defining the sketch geometry
dSize = swEndPoint.X - swStartPoint.X
dFactor1 = 0.1
dFactor2 = 1.25

swModel.SetAddToDB(True)
swModel.SetDisplayWhenAdded(False)

swModel.SketchManager.CreateLine(swStartPoint.X, swStartPoint.Y, 0.0#, swStartPoint.X, swStartPoint.Y + dLength, 0.0#)
swModel.SketchManager.CreateLine(swStartPoint.X, swStartPoint.Y + dLength, 0.0#, swStartPoint.X + dFactor1 * dSize,
swStartPoint.Y + dFactor2 * dLength, 0.0#)
swModel.SketchManager.CreateLine(swStartPoint.X + dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0#, swEndPoint.X
- dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0#)
swModel.SketchManager.CreateLine(swEndPoint.X - dFactor1 * dSize, swStartPoint.Y + dFactor2 * dLength, 0.0#, swEndPoint.X,
swEndPoint.Y + dLength, 0.0#)
swModel.SketchManager.CreateLine(swEndPoint.X, swEndPoint.Y, 0.0#, swEndPoint.X, swEndPoint.Y + dLength, 0.0#)

' Reset
swModel.SetDisplayWhenAdded(True)
swModel.SetAddToDB(False)

' Commit changes made to the sketch
swModel.SketchManager.InsertSketch(True)

' Set options
nOptions = swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRadius +
swInsertEdgeFlangeOptions_e.swInsertEdgeFlangeUseDefaultRelief

aFlangeEdges(0) = swEdge
aSketchFeats(0) = swSketch

vFlangeEdges = aFlangeEdges
vSketchFeats = aSketchFeats

swFeature = swModel.FeatureManager.InsertSheetMetalEdgeFlange2((vFlangeEdges), (vSketchFeats), nOptions, dAngle, 0.0#,
swFlangePositionTypes_e.swFlangePositionTypeBendOutside, dLength, swSheetMetalReliefTypes_e.swSheetMetalReliefNone, 0.0#, 0.0#,
0.0#, swFlangeDimTypes_e.swFlangeDimTypeInnerVirtualSharp, Nothing)

' Rotate view so that IModelDocExtension::SelectByID2 coordinates can be found
Dim myModelView As ModelView
myModelView = swModel.ActiveView
myModelView.RotateAboutCenter(45, 0.00911235438195936)

' Select the sheet metal body to which to apply a corner relief
bValue = swModel.Extension.SelectByID2("Edge-Flange1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
swModel.ClearSelection2(True)

' Specify two corners of the edge flange for which to create a corner relief

' Select faces that define the first corner
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0549242492243928, 0.053073918098565, 0.0242634000000049, True, 4,
Nothing, 0)
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0276778697571744, 0.0530739180985651, -0.00104170971004399, True, 4,
Nothing, 0)
Dim myCorner As Long
myCorner = swModel.FeatureManager.AddCornerReliefCorner()

' Specify the type of corner relief to apply to the first corner
Dim myReliefType As Boolean
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, swCornerReliefType_e.swCornerSquareRelief, 0.0001, 0.0007366,
0.00018415, False, False, False, False)
swModel.ClearSelection2(True)

' Select faces that define the second corner
bValue = swModel.Extension.SelectByID2("", "FACE", 0.0276778697571744, 0.0530739180985651, -0.00104170971004399, True, 4,
Nothing, 0)
bValue = swModel.Extension.SelectByID2("", "FACE", 0.000431490289955978, 0.053073918098565, 0.0242634000000049, True, 4,
Nothing, 0)
myCorner = swModel.FeatureManager.AddCornerReliefCorner()

' Specify the type of corner relief to apply to the second corner
myReliefType = swModel.FeatureManager.AddCornerReliefType(-1, swCornerReliefType_e.swCornerObroundRelief, 0.0001,
0.0029464, 0.0007366, False, False, False, False)

' Create the corner relief feature
Dim myFeature As Feature
myFeature = swModel.FeatureManager.FinishCornerRelief()

End Sub

Public swApp As SldWorks

End Class

```


Create_Countersink_Slot_Example_CSharp (Topic)

SOLIDWORKS API Help

Create Countersink Slot Example (C#)

This example shows how to create a countersink slot using the hole wizard.

```
//-----
// Preconditions:
// 1. SOLIDWORKS Design is running.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Creates a model and a countersink
//    slot in the model using the hole wizard.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace HoleWizard5FeatureManagerCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            SketchManager swSketchMgr = default(SketchManager);
            WizardHoleFeatureData2 swWizardHoleFeatData = default(WizardHoleFeatureData2);
            object sketchLines = null;
            int status = 0;
            bool boolstatus = false;
            double[] P1 = new double[3];
            double[] P2 = new double[3];
            double[] P3 = new double[3];

            // Create the model for the countersink slot
            swApp.ResetUntitledCount(0, 0, 0);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\Documents and Settings\\All Users\\Application Data\\SOLIDWORKS\\SOLIDWORKS 2014\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref status);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSketchMgr = swModel.SketchManager;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;
            sketchLines = swSketchMgr.CreateCornerRectangle(-0.05096498314664, 0.05060941349678, 0, 0.1021670127265, -0.05037236706354, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeat = swFeatMgr.FeatureExtrusion2(true, false, false, 0, 0, 0.381, 0.381, false, false, false,
            false, 0.01745329251994, 0.01745329251994, false, false, false, false, true, true, true,
            0, 0, false);

            //Create three points for the reference plane
            P1[0] = -0.0141556764402858;
            P1[1] = 0.00194061273859598;
            P1[2] = 0;
            P2[0] = -0.0141556764402858;
            P2[1] = 0.00194061273859598;
            P2[2] = 1;
            P3[0] = -0.149976101832345;
            P3[1] = -0.988792859011662;
            P3[2] = 0;

            //Create the reference plane
            swModel.CreatePlaneFixed2(P1, P2, P3, false);
            //Select reference plane
            boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", -0.0156784487003801, -0.00916715285390111, 0.0558270998665543, false, 0, null, 0);
            // Create the countersink slot
            swFeat = swFeatMgr.HoleWizard5((int)swWzdGeneralHoleTypes_e.swWzdCounterSinkSlot, (int)swWzdHoleStandards_e.swStandardAnsiMetric, (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiMetricFlatHead
            82, "M2", (int)swEndConditions_e.swEndCondThroughAll, 0.0102, 0.010312189893273, 1, 0.0044, 1.57079632679489, 0.000152189893272978,
            0, 2.05948851735331, 0, 0, 0, 1, 0, 0, 0, "",
            false, true, true, true, true, false);

            swWizardHoleFeatData = (WizardHoleFeatureData2)swFeat.GetDefinition;
            Debug.Print("Length of countersink slot: " + swWizardHoleFeatData.Length;

        }
    }
}
```

Create Countersink Slot Example (VBA)

This example shows how to create a countersink slot using the hole wizard.

```
'-----
' Preconditions:
' 1. SOLIDWORKS Design is running.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Creates a model and a countersink
'    slot in the model using the hole wizard.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swSketchMgr As SldWorks.SketchManager

Dim swWizardHoleFeatData As SldWorks.WizardHoleFeatureData2
Dim sketchLines As Variant
Dim longstatus As Long
Dim boolstatus As Boolean
Dim P1(2) As Double
Dim P2(2) As Double
Dim P3(2) As Double
```

Sub main()

```
    Set swApp = Application.SldWorks
```

```
    ' Create the model for the countersink slot
```

```
    swApp.ResetUntitledCount 0, 0, 0
```

```
    Set swModel = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS
2014\templates\Part.prtdot", 0, 0, 0)
```

```
    swApp.ActivateDoc2 "Part1", False, longstatus
```

```
    Set swModel = swApp.ActiveDoc
```

```
    Set swSketchMgr = swModel.SketchManager
```

```
    Set swModelDocExt = swModel.Extension
```

```
    Set swFeatMgr = swModel.FeatureManager
```

```
    sketchLines = swSketchMgr.CreateCornerRectangle(-0.05096498314664, 0.05060941349678, 0, 0.1021670127265,
-0.05037236706354, 0)
```

```
    boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
```

```
    boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
    boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
    boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
    Set swFeat = swFeatMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.381, 0.381, False, False, False, False,
0.01745329251994, 0.01745329251994, False, False, False, False, True, True, True, 0, 0, False)
```

```
    'Create three points for the reference plane
```

```
    P1(0) = -1.41556764402858E-02
```

```
    P1(1) = 1.94061273859598E-03
```

```
    P1(2) = 0
```

```
    P2(0) = -1.41556764402858E-02
```

```
    P2(1) = 1.94061273859598E-03
```

```
    P2(2) = 1
```

```
    P3(0) = -0.149976101832345
```

```
    P3(1) = -0.988792859011662
```

```
    P3(2) = 0
```

'Create the reference plane

swModel.**CreatePlaneFixed2** P1, P2, P3, False

'Select reference plane

boolstatus = swModelDocExt.**SelectByID2**("Plane1", "PLANE", -1.56784487003801E-02, -9.16715285390111E-03, 5.58270998665543E-02, False, 0, Nothing, 0)

' Create the hole countersink slot

Set swFeat = swFeatMgr.**HoleWizard5**(swWzdCounterSinkSlot, swStandardAnsiMetric, swStandardAnsiMetricFlatHead82, "M2", swEndCondThroughAll, 0.0102, 0.010312189893273, _

1, 0.0044, 1.57079632679489, 1.52189893272978E-04, 0, 2.05948851735331, 0, 0, 0, 1, 0, 0, 0, "", False, True, True, True, True, False)

Set swWizardHoleFeatData = swFeat.**GetDefinition**

Debug.Print ("Length of countersink slot: " & swWizardHoleFeatData.**Length**)

End Sub

Create Countersink Slot Example (VB.NET)

This example shows how to a countersink slot using the hole wizard.

```
'-----
' Preconditions:
' 1. SOLIDWORKS Design is running.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Creates a model and a countersink
'    slot in the model using the hole wizard.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swSketchMgr As SketchManager
        Dim swWizardHoleFeatData As WizardHoleFeatureData2
        Dim sketchLines As Object
        Dim status As Integer
        Dim boolstatus As Boolean
        Dim P1(2) As Double
        Dim P2(2) As Double
        Dim P3(2) As Double

        ' Create the model for the countersink slot
        swModel.ResetUntitledCount(0, 0, 0)
        swModel = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2014\templates\Part.prt", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, status)
        swModel = swApp.ActiveDoc
        swSketchMgr = swModel.SketchManager
        swModelDocExt = swModel.Extension
        swFeatMgr = swModel.FeatureManager
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.05096498314664, 0.05060941349678, 0, 0.1021670127265, -0.05037236706354, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeat = swFeatMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.381, 0.381, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, True, True, True, 0, 0, False)

        'Create three points for the reference plane
        P1(0) = -0.0141556764402858
        P1(1) = 0.00194061273859598
        P1(2) = 0
        P2(0) = -0.0141556764402858
        P2(1) = 0.00194061273859598
        P2(2) = 1
        P3(0) = -0.149976101832345
        P3(1) = -0.988792859011662
        P3(2) = 0

        'Create the reference plane
        swModel.CreatePlaneFixed2(P1, P2, P3, False)

        'Select the reference plane
        boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", -0.0156784487003801, -0.00916715285390111, 0.0558270998665543, False, 0, Nothing, 0)

        ' Create the countersink slot
        swFeat = swFeatMgr.HoleWizard5(swWzdGeneralHoleTypes_e.swWzdCounterSinkSlot, swWzdHoleStandards_e.swStandardAnsiMetric, swWzdHoleStandardFastenerTypes_e.swStandardAnsiMetricFlatHead82, "M2", swEndConditions_e.swEndCondThroughAll, 0.0102, 0.010312189893273, 1, 0.0044, 1.57079632679489, 0.000152189893272978, 0, 2.05948851735331, 0, 0, 0, 1, 0, 0, 0, "", False, True, True, True, True, False)

        swWizardHoleFeatData = swFeat.GetDefinition
        Debug.Print("Length of countersink slot: " & swWizardHoleFeatData.Length)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Create Curvature Continuous Variable Fillet Feature Example (C#)

This example shows how to create a curvature continuous variable fillet feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a part with a variable fillet feature with curvature continuous.
// 2. Gets whether the variable fillet feature has curvature continuous set.
// 3. Examine the graphics area and the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            VariableFilletFeatureData2 swVariableFilletFeatureData = default(VariableFilletFeatureData2);
            bool status = false;
            object radiiArray2 = null;
            double[] radiis2 = null;
            object dist2Array2 = null;
            double[] dists22 = null;
            object conicRhosArray2 = null;
            double[] conirhos2 = null;
            object setBackArray2 = null;
            double setBacks2 = 0;
            object pointArray2 = null;
            double points2 = 0;
            object pointDist2Array2 = null;
            double pointsDist22 = 0;
            object pointRhoArray2 = null;
            double pointsRhos2 = 0;
            int filletOptions = 0;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Create part with variable feature fillet with curvature continuous
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            swSketchSegment = swSketchManager.CreateLine(-0.070405, 0.03105, 0.0, -0.070405, -0.033217, 0.0);
            swSketchSegment = swSketchManager.CreateLine(-0.070405, -0.033217, 0.0, 0.01733, -0.033217, 0.0);
            swSketchSegment = swSketchManager.CreateLine(0.01733, -0.033217, 0.0, 0.01733, -0.016247, 0.0);
            swSketchSegment = swSketchManager.CreateLine(0.01733, -0.016247, 0.0, -0.026537, -0.009748, 0.0);
            swSketchSegment = swSketchManager.CreateLine(-0.026537, -0.009748, 0.0, -0.04116, 0.024552, 0.0);
            swSketchSegment = swSketchManager.CreateLine(-0.04116, 0.024552, 0.0, -0.070405, 0.03105, 0.0);
            swModel.ShowNamedView2("Trimetric", 8);
            swModel.ClearSelection2(true);
            swFeature = (Feature)swFeatureManager.FeatureExtrusion3(true, false, false, 0, 0, 0.04064, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);
            swSelectionMgr.EnableContourSelection = false;
            status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 0.0243713283917941, 0.023466279649881, true, 1, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 0.0243713283917941, 0.023466279649881, false, 1, null, 0);
            radiis2 = new double[2];
            radiis2[0] = 0.00508;
            radiis2[1] = 0.01778;
            dists22 = new double[2];
            conirhos2 = new double[2];
            radiiArray2 = radiis2;
            dist2Array2 = dists22;
            conicRhosArray2 = conirhos2;
            setBackArray2 = setBacks2;
            pointArray2 = points2;
            pointDist2Array2 = pointsDist22;
            pointRhoArray2 = pointsRhos2;

            filletOptions = (int)swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous + (int)swFeatureFilletOptions_e.swFeatureFilletKeepFeatures +
            (int)swFeatureFilletOptions_e.swFeatureFilletPropagateFeatToParts;
            swFeature = (Feature)swFeatureManager.FeatureFillet3(filletOptions, 0.00254, 1E-
            07, 0, 1, 0, 0, (radiiArray2), (dist2Array2), (conicRhosArray2),
            (setBackArray2), (pointArray2), (pointDist2Array2), (pointRhoArray2));

            //Verify variable fillet feature has curvature continuous set
            swVariableFilletFeatureData = (VariableFilletFeatureData2)swFeature.GetDefinition();
            status = swVariableFilletFeatureData.AccessSelections(swModel, null);
            Debug.Print("Variable fillet feature has continuous curvature set: " + swVariableFilletFeatureData.CurvatureContinuous);
            swVariableFilletFeatureData.ReleaseSelectionAccess();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Create Curvature Continuous Variable Fillet Feature Example (VBA)

This example shows how to create a curvature continuous variable fillet feature.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a part with a variable fillet feature with curvature continuous.
' 2. Gets whether the variable fillet feature has curvature continuous set.
' 3. Examine the graphics area and the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swVariableFilletFeatureData As SldWorks.VariableFilletFeatureData2
Dim status As Boolean
Dim radiiArray2 As Variant
Dim radii2() As Double
Dim dist2Array2 As Variant
Dim dists22() As Double
Dim conicRhosArray2 As Variant
Dim conIRhos2() As Double
Dim setBackArray2 As Variant
Dim setBacks2 As Double
Dim pointArray2 As Variant
Dim points2 As Double
Dim pointDist2Array2 As Variant
Dim pointsDist22 As Double
Dim pointRhoArray2 As Variant
Dim pointsRhos2 As Double
Dim filletOptions As Long

Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchManager = swModel.SketchManager
    Set swFeatureManager = swModel.FeatureManager
    Set swSelectionMgr = swModel.SelectionManager

    'Create part with variable feature fillet with curvature continuous
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchManager.CreateLine(-0.070405, 0.03105, 0#, -0.070405, -0.033217, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.070405, -0.033217, 0#, 0.01733, -0.033217, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.01733, -0.033217, 0#, 0.01733, -0.016247, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.01733, -0.016247, 0#, -0.026537, -0.009748, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.026537, -0.009748, 0#, -0.04116, 0.024552, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.04116, 0.024552, 0#, -0.070405, 0.03105, 0#)
    swModel.ShowNamedView2 "Trimetric", 8
    swModel.ClearSelection2 True
    Set swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.04064, 0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, True, True, True, 0, 0, False)
    swSelectionMgr.EnableContourSelection = False
    status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 2.43713283917941E-02, 0.023466279649881, True, 1, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 2.43713283917941E-02, 0.023466279649881, False, 1, Nothing, 0)

    ReDim radii2(0 To 1) As Double
    radii2(0) = 0.00508
    radii2(1) = 0.01778

    ReDim dists22(0 To 1) As Double
    ReDim conIRhos2(0 To 1) As Double
    radiiArray2 = radii2
    dist2Array2 = dists22
    conicRhosArray2 = conIRhos2
    setBackArray2 = setBacks2
    pointArray2 = points2
    pointDist2Array2 = pointsDist22
    pointRhoArray2 = pointsRhos2

    filletOptions = swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous + swFeatureFilletOptions_e.swFeatureFilletKeepFeatures + swFeatureFilletOptions_e.swFeatureFilletPropagateFeatToParts
    Set swFeature = swFeatureManager.FeatureFillet3(filletOptions, 0.00254, 0.0000001, 0, 1, 0, 0, (radiiArray2), (dist2Array2), (conicRhosArray2), (setBackArray2), (pointArray2), (pointDist2Array2), (pointRhoArray2))

    'Verify variable fillet feature has curvature continuous set
    Set swVariableFilletFeatureData = swFeature.GetDefinition
    status = swVariableFilletFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Variable fillet feature has curvature continuous set: " & swVariableFilletFeatureData.CurvatureContinuous
    swVariableFilletFeatureData.ReleaseSelectionAccess
```

End Sub

Create Curvature Continuous Variable Fillet Feature Example (VB.NET)

This example shows how to create a curvature continuous variable fillet feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a part with a variable fillet feature with curvature continuous.
' 2. Gets whether the variable fillet feature has curvature continuous set.
' 3. Examine the graphics area and the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swFeatureManager As FeatureManager
        Dim swSelectionMgr As SelectionMgr
        Dim swVariableFilletFeatureData As VariableFilletFeatureData2
        Dim status As Boolean
        Dim radiiArray2 As Object
        Dim radii2() As Double
        Dim dist2Array2 As Object
        Dim dists22() As Double
        Dim conicRhosArray2 As Object
        Dim conicRhos2() As Double
        Dim setBackArray2 As Object
        Dim setBacks2 As Double
        Dim pointArray2 As Object
        Dim points2 As Double
        Dim pointDist2Array2 As Object
        Dim pointsDist22 As Double
        Dim pointRhoArray2 As Object
        Dim pointsRhos2 As Double
        Dim filletOptions As Integer

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchManager = swModel.SketchManager
        swFeatureManager = swModel.FeatureManager
        swSelectionMgr = swModel.SelectionManager

        'Create part with variable feature fillet with curvature continuous
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateLine(-0.070405, 0.03105, 0.0#, -0.070405, -0.033217, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.070405, -0.033217, 0.0#, 0.01733, -0.033217, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.01733, -0.033217, 0.0#, 0.01733, -0.016247, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.01733, -0.016247, 0.0#, -0.026537, -0.009748, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.026537, -0.009748, 0.0#, -0.04116, 0.024552, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.04116, 0.024552, 0.0#, -0.070405, 0.03105, 0.0#)
        swModel.ShowNamedView2("Trimetric", 8)
        swModel.ClearSelection2(True)
        swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.04064, 0.00254, False, False, False, False, 0.01
74532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionMgr.EnableContourSelection = False
        status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 0.0243713283917941, 0.023466279649881, True, 1, Nothing,
0)

        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "EDGE", -0.040927911364804, 0.0243713283917941, 0.023466279649881, False, 1, Nothing
, 0)

        ReDim radii2(1)
        radii2(0) = 0.00508
        radii2(1) = 0.01778
        ReDim dists22(1)
        ReDim conicRhos2(1)
        radiiArray2 = radii2
        dist2Array2 = dists22
        conicRhosArray2 = conicRhos2
        setBackArray2 = setBacks2
        pointArray2 = points2
        pointDist2Array2 = pointsDist22
        pointRhoArray2 = pointsRhos2

        filletOptions = swFeatureFilletOptions_e.swFeatureFilletCurvatureContinuous + swFeatureFilletOptions_e.swFeatureFilletKeepF

```

```

eatures + swFeatureFilletOptions_e.swFeatureFilletPropagateFeatToParts
    swFeature = swFeatureManager.FeatureFillet3(filletOptions, 0.00254, 0.0000001, 0, 1, 0, 0, (radiiArray2), (dist2Array2), (c
onicRhosArray2), (setBackArray2), (pointArray2), (pointDist2Array2), (pointRhoArray2))

    'Verify variable fillet feature has curvature continuous set
    swVariableFilletFeatureData = swFeature.GetDefinition
    status = swVariableFilletFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print("Variable fillet feature has curvature continuous set: " & swVariableFilletFeatureData.CurvatureContinuous)
    swVariableFilletFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create Curve Through Reference Points Example (C#)

This example shows how to create a curve through reference points.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Opens the part document.
// 2. Selects four reference points.
// 3. Creates Curve1 through the selected points.
// 4. Inspect the FeatureManager design tree, the graphics area, and the
//    Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace InsertCurveThroughPoints_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature feat;
        ReferencePointCurveFeatureData featData;
        SelectionMgr selMgr;
        bool boolstatus;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\block20.sldprt", 1, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("block20", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;
            selMgr = (SelectionMgr)Part.SelectionManager;

            boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.0646002912861796, 0, 0.0493456023787655, false, 1, null, 0);
            boolstatus = Part.Extension.SelectByID2("", "VERTEX", 0.0646002912861796, 0.039624, 0.0493456023787655, true, 1, null, 0);
            boolstatus = Part.Extension.SelectByID2("", "VERTEX", -0.0624778997860176, 0.039624, 0.0493456023787655, true, 1, null, 0);
            boolstatus = Part.Extension.SelectByID2("", "VERTEX", -0.0624778997860176, 0, 0.0493456023787655, true, 1, null, 0);

            Part.Insert3DSplineCurve(false);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, false, 0, null, 0);
            feat = (Feature)selMgr.GetSelectedObject6(1, -1);
            featData = (ReferencePointCurveFeatureData)feat.GetDefinition();

            featData.AccessSelections(Part, null);
            Debug.Print(feat.Name);
            Debug.Print("  Closed curve? " + featData.ClosedCurve);
            Debug.Print("  Number of through points: " + featData.GetThroughPointCount());

            featData.ReleaseSelectionAccess();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Create Curve Through Reference Points Example (VBA)

This example shows how to create a curve through reference points.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the part document.
' 2. Selects four reference points.
' 3. Creates Curve1 through the selected points.
' 4. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim feat As SldWorks.Feature
Dim featData As SldWorks.ReferencePointCurveFeatureData
Dim selMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt", 1, 0, "",
longstatus, longwarnings)
    swApp.ActivateDoc2 "block20", False, longstatus
    Set Part = swApp.ActiveDoc
    Set selMgr = Part.SelectionManager

    boolstatus = Part.Extension.SelectByID2("", "VERTEX", 6.46002912861796E-02, 0, 4.93456023787655E-02, False, 1, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("", "VERTEX", 6.46002912861796E-02, 0.039624, 4.93456023787655E-02, True, 1, Nothing,
0)
    boolstatus = Part.Extension.SelectByID2("", "VERTEX", -6.24778997860176E-02, 0.039624, 4.93456023787655E-02, True, 1, Nothing,
0)
    boolstatus = Part.Extension.SelectByID2("", "VERTEX", -6.24778997860176E-02, 0, 4.93456023787655E-02, True, 1, Nothing, 0)

    Part.Insert3DSplineCurve False
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
    Set feat = selMgr.GetSelectedObject6(1, -1)
    Set featData = feat.GetDefinition

    featData.AccessSelections Part, Nothing
    Debug.Print feat.Name
    Debug.Print " Closed curve? " & featData.ClosedCurve
    Debug.Print " Number of through points: " & featData.GetThroughPointCount

    featData.ReleaseSelectionAccess

End Sub

```

Create Curve Through Reference Points Example (VB.NET)

This example shows how to create a curve through reference points.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the part document.
' 2. Selects four reference points.
' 3. Creates Curve1 through the selected points.
' 4. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim Part As ModelDoc2
    Dim feat As Feature
    Dim featData As ReferencePointCurveFeatureData
    Dim selMgr As SelectionMgr
    Dim boolstatus As Boolean
    Dim longstatus As Integer, longwarnings As Integer

    Sub main()

        Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\block20.sldprt", 1, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("block20", False, longstatus)
        Part = swApp.ActiveDoc
        selMgr = Part.SelectionManager

        boolstatus = Part.Extension.SelectByID("", "VERTEX", 0.0646002912861796, 0, 0.0493456023787655, False, 1, Nothing, 0)
        boolstatus = Part.Extension.SelectByID("", "VERTEX", 0.0646002912861796, 0.039624, 0.0493456023787655, True, 1, Nothing, 0)
        boolstatus = Part.Extension.SelectByID("", "VERTEX", -0.0624778997860176, 0.039624, 0.0493456023787655, True, 1, Nothing, 0)
        boolstatus = Part.Extension.SelectByID("", "VERTEX", -0.0624778997860176, 0, 0.0493456023787655, True, 1, Nothing, 0)

        Part.Insert3DSplineCurve(False)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID("Curve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
        feat = selMgr.GetSelectedObject6(1, -1)
        featData = feat.GetDefinition

        featData.AccessSelections(Part, Nothing)
        Debug.Print(feat.Name)
        Debug.Print(" Closed curve? " & featData.ClosedCurve)
        Debug.Print(" Number of through points: " & featData.GetThroughPointCount)

        featData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Create Cut-sweep Feature Using Circular Profile Example (C#)

This example shows how to create a cut-sweep feature using a circular profile.

```
//-----
// Preconditions:
// 1. Verify that the part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects an edge on the part.
// 3. Creates a cut-sweep feature using a circular
//    profile.
// 4. Accesses the cut-sweep feature.
// 5. Changes the diameter of the circular profile.
// 6. Examine the Immediate window, FeatureManager
//    design tree, and graphics area.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            SweepFeatureData swSweepFeatureData = default(SweepFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\block20.sldprt", 1, 0, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select the edge for the cut-sweep feature
            status = swModelDocExt.SelectByID2("", "EDGE", -0.0372983839416747, 0.0393904284381961, 0.0495042874504179, true, 4, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            //Create the cut-sweep feature using a circular profile
            swFeature = (Feature)swFeatureManager.InsertCutSwept5(false, true, 0, false, false, 0, 0, false, 0, 0,
            0, 0, true, true, 0, true, true, true, false, true,
            0.0254, (int)swSweepDirection_e.swSweepDirection1);

            //Roll back the cut-sweep feature to access and change the diameter of circular profile
            swSweepFeatureData = (SweepFeatureData)swFeature.GetDefinition();
            status = swSweepFeatureData.AccessSelections(swModel, null);
            Debug.Print("Using a circular profile? " + swSweepFeatureData.CircularProfile);
            Debug.Print("Original size of circular profile = " + swSweepFeatureData.CircularProfileDiameter);
            swSweepFeatureData.CircularProfileDiameter = 0.003;
            Debug.Print("Modified size of circular profile = " + swSweepFeatureData.CircularProfileDiameter);
            Debug.Print("Direction of cut-sweep = " + swSweepFeatureData.Direction);

            //Roll forward the cut-sweep feature
            swFeature.ModifyDefinition(swSweepFeatureData, swModel, null);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Cut-sweep Feature Using Circular Profile Example (VBA)

This example shows how to create a cut-sweep feature using a circular profile.

```
'-----
' Preconditions:
' 1. Verify that the part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects an edge on the part.
' 3. Creates a cut-sweep feature using a circular
'    profile.
' 4. Accesses the cut-sweep feature.
' 5. Changes the diameter of the circular profile.
' 6. Examine the Immediate window, FeatureManager
'    design tree, and graphics area.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSweepFeatureData As SldWorks.SweepFeatureData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt", 1, 0, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Select the edge for the cut-sweep feature
    status = swModelDocExt.SelectByID("", "EDGE", -3.72983839416747E-02, 3.93904284381961E-02, 4.95042874504179E-02, True, 4, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager

    'Create the cut-sweep feature using a circular profile
    Set swFeature = swFeatureManager.InsertCutSwept5(False, True, 0, False, False, 0, 0, False, 0, 0, 0, 0, True, True, 0, True, True, True, False, True,
    0.0254, swSweepDirection_e.swSweepDirection1)

    'Roll back the cut-sweep feature to access and change the diameter of circular profile
    Set swSweepFeatureData = swFeature.GetDefinition
    status = swSweepFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Using a circular profile? " & swSweepFeatureData.CircularProfile
    Debug.Print "Original size of circular profile = " & swSweepFeatureData.CircularProfileDiameter
    swSweepFeatureData.CircularProfileDiameter = 0.003
    Debug.Print "Modified size of circular profile = " & swSweepFeatureData.CircularProfileDiameter
    Debug.Print "Direction of cut-sweep = " & swSweepFeatureData.Direction

    'Roll forward the cut-sweep feature
    swFeature.ModifyDefinition swSweepFeatureData, swModel, Nothing

End Sub
```

Create Cut-sweep Feature Using Circular Profile Example (VB.NET)

This example shows how to create a cut-sweep feature using a circular profile.

```

-----
Preconditions:
1. Verify that the part exists.
2. Open the Immediate window.

Postconditions:
1. Opens the part.
2. Selects an edge on the part.
3. Creates a cut-sweep feature using a circular profile.
4. Accesses the cut-sweep feature.
5. Changes the diameter of the circular profile.
6. Examine the Immediate window, FeatureManager design tree, and graphics area.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swFeatureManager As FeatureManager
        Dim swSweepFeatureData As SweepFeatureData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt", 1, 0, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Select the edge for the cut-sweep feature
        status = swModelDocExt.SelectByID("", "EDGE", -0.0372983839416747, 0.0393904284381961, 0.0495042874504179, True, 4, Nothing, 0)
        swFeatureManager = swModel.FeatureManager

        'Create the cut-sweep feature using a circular profile
        swFeature = swFeatureManager.InsertCutSwept5(False, True, 0, False, False, 0, 0, False, 0, 0, 0, 0, True, True, 0, True, True, True, False, True,
0.0254, swSweepDirection_e.swSweepDirection1)

        'Roll back the cut-sweep feature to access and change the diameter of circular profile
        swSweepFeatureData = swFeature.GetDefinition
        status = swSweepFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Using a circular profile? " & swSweepFeatureData.CircularProfile)
        Debug.Print("Original size of circular profile = " & swSweepFeatureData.CircularProfileDiameter)
        swSweepFeatureData.CircularProfileDiameter = 0.003
        Debug.Print("Modified size of circular profile = " & swSweepFeatureData.CircularProfileDiameter)
        Debug.Print("Direction of cut-sweep = " & swSweepFeatureData.Direction)

        'Roll forward the cut-sweep feature
        swFeature.ModifyDefinition(swSweepFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Cut-sweep Feature Using Tool Body Example (C#)

This example shows how to create a cut-sweep feature using a tool body.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a boss-extrude feature.
// 2. Creates a sketch.
// 3. Creates a revolve feature.
// 4. Selects the revolve feature, sketch, and extrude feature and
//    creates a cut-sweep feature.
// 5. Accesses the cut-sweep feature.
// 6. Gets the names of the cut-sweep feature's tool body and path.
// 7. Releases access of the cut-sweep feature.
// 8. Examine the Immediate window, FeatureManager design tree,
//    and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            SweepFeatureData swSweepFeatureData = default(SweepFeatureData);
            object swProfileObj = null;
            Body2 swProfileBody = default(Body2);
            Feature swPathFeature = default(Feature);
            object[] sketchLines = null;
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create extrude feature
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.000361, 0.001416, 0.0, 0.024462, -0.045092, 0.0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, true, 0, 0, 0.09, 0.01, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);

            //Create sketch
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swSelectionMgr.EnableContourSelection = false;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-1.9E-05, 0.00051, 0.0, 0.026716, -0.0401, 0.0);
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Create revolve feature
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.0266210577384013, -0.0248555003438298, 0, -0.0378465609175683, -0.047510606759966
            9, 0);

            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", -0.0264169576805983, -0.0449999999999999, 0.0293457016154969, false, 16, null, 0);

            swFeature = (Feature)swFeatureMgr.FeatureRevolve2(true, true, false, false, false, false, 0, 0, 6.2831853071796, 0,
            false, false, 0.01, 0.01, 0, 0, false, true, true);
            swSelectionMgr.EnableContourSelection = false;
            swModel.ClearSelection2(true);

            //Create cut-sweep feature
            status = swModelDocExt.SelectByID2("Revolve1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Revolve1", "SOLIDBODY", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, true, 2048, null, 0);
            swFeature = (Feature)swFeatureMgr.InsertCutSweep5(false, true, 0, false, false, 0, 0, false, 0, 0,
            0, 0, true, false, 0, true, true, true, false, false,
            0, 0);
            Debug.Print("Feature name = " + swFeature.Name);
            swSweepFeatureData = (SweepFeatureData)swFeature.GetDefinition();

            // Roll back to access selections
            status = swSweepFeatureData.AccessSelections(swModel, null);
            swProfileObj = (object)swSweepFeatureData.Profile;
            swProfileBody = (Body2)swProfileObj;
            Debug.Print(" Tool body = " + swProfileBody.Name);
            swPathFeature = (Feature)swSweepFeatureData.Path;
            Debug.Print(" Path = " + swPathFeature.Name);

            // Roll forward
            swSweepFeatureData.ReleaseSelectionAccess();
        }
    }
}
```

```
    /// <summary>  
    /// The Sldworks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Create Cut-sweep Feature Using Tool Body Example (VBA)

This example shows how to create a cut-sweep feature using a tool body.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a boss-extrude feature.
' 2. Creates a sketch.
' 3. Creates a revolve feature.
' 4. Selects the revolve feature, sketch, and extrude feature and
'    creates a cut-sweep feature.
' 5. Accesses the cut-sweep feature.
' 6. Gets the names of the cut-sweep feature's tool body and path.
' 7. Releases access of the cut-sweep feature.
' 8. Examines the Immediate window, FeatureManager design tree,
'    and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swSweepFeatureData As SldWorks.SweepFeatureData
Dim swProfileObj As Object
Dim swProfileBody As SldWorks.Body2
Dim swPathFeature As SldWorks.Feature
Dim sketchLines As Variant
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create extrude feature
    status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.000361, 0.001416, 0#, 0.024462, -0.045092, 0#)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, True, 0, 0, 0.09, 0.01, False, False, False, False, 1.74532925199433E-02,
    1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)

    'Create sketch
    status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    swSelectionMgr.EnableContourSelection = False
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.000019, 0.00051, 0#, 0.026716, -0.0401, 0#)
    swSketchMgr.InsertSketch True

    swModel.ClearSelection2 True

    'Create revolve feature
    status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchLines = swSketchMgr.CreateCornerRectangle(-2.66210577384013E-02, -2.48555003438298E-02, 0, -3.78465609175683E-02, -4.75106067599669E-02, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", -2.64169576805983E-02, -4.49999999999998E-02, 2.93457016154969E-02, False, 16, Nothing,
    0)
    Set swFeature = swFeatureMgr.FeatureRevolve2(True, True, False, False, False, False, 0, 0, 6.2831853071796, 0, False, False, 0.01, 0.01, 0, 0, 0,
    False, True, True)
    swSelectionMgr.EnableContourSelection = False

    swModel.ClearSelection2 True

    'Create cut-sweep feature
    status = swModelDocExt.SelectByID("Revolve1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Revolve1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 2048, Nothing, 0)
    Set swFeature = swFeatureMgr.InsertCutSwept5(False, True, 0, False, False, 0, 0, False, 0, 0, 0, True, False, 0, True, True, True, False, False, 0,
    0)

    Debug.Print "Feature name = " & swFeature.Name
    Set swSweepFeatureData = swFeature.GetDefinition

    ' Roll back to access selections
    status = swSweepFeatureData.AccessSelections(swModel, Nothing)

    Set swProfileObj = swSweepFeatureData.Profile
    Set swProfileBody = swProfileObj
    Debug.Print " Tool body = " & swProfileBody.Name

```

```
Set swPathFeature = swSweepFeatureData.Path
Debug.Print " Path = " & swPathFeature.Name

' Roll forward
swSweepFeatureData.ReleaseSelectionAccess

End Sub
```

Create Cut-sweep Feature Using Tool Body Example (VB.NET)

This example shows how to create a cut-sweep feature using a tool body.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Creates a boss-extrude feature.
2. Creates a sketch.
3. Creates a revolve feature.
4. Selects the revolve feature, sketch, and extrude feature and
   creates a cut-sweep feature.
5. Accesses the cut-sweep feature.
6. Gets the names of the cut-sweep feature's tool body and path.
7. Releases access of the cut-sweep feature.
8. Examine the Immediate window, FeatureManager design tree,
   and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swFeatureMgr As FeatureManager
        Dim swSelectionMgr As SelectionMgr
        Dim swSweepFeatureData As SweepFeatureData
        Dim swProfileObj As Object
        Dim swProfileBody As Body2
        Dim swPathFeature As Feature
        Dim sketchLines As Object
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Create extrude feature
        status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateCircle(-0.000361, 0.001416, 0.0#, 0.024462, -0.045092, 0.0#)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, True, 0, 0, 0.09, 0.01, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)

        'Create sketch
        status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swSelectionMgr.EnableContourSelection = False
        swSketchSegment = swSketchMgr.CreateCircle(-0.000019, 0.00051, 0.0#, 0.026716, -0.0401, 0.0#)
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)

        'Create revolve feature
        status = swModelDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.0266210577384013, -0.0248555003438298, 0, -0.0378465609175683, -0.0475106067599669, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", -0.0264169576805983, -0.0449999999999999, 0.0293457016154969, False, 16, Nothing, 0)
        swFeature = swFeatureMgr.FeatureRevolve2(True, True, False, False, False, False, 0, 0, 6.2831853071796, 0, False, False, 0.01, 0.01, 0, 0, 0, False, e, True, True)
        swSelectionMgr.EnableContourSelection = False
        swModel.ClearSelection2(True)

        'Create cut-sweep feature
        status = swModelDocExt.SelectByID("Revolve1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Revolve1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 2048, Nothing, 0)
        swFeature = swFeatureMgr.InsertCutSwept5(False, True, 0, False, False, 0, 0, False, 0, 0, 0, 0, True, False, 0, True, True, True, False, False, 0, 0)

        Debug.Print("Feature name = " & swFeature.Name)
        swSweepFeatureData = swFeature.GetDefinition

        ' Roll back to access selections
        status = swSweepFeatureData.AccessSelections(swModel, Nothing)
        swProfileObj = swSweepFeatureData.Profile
        swProfileBody = swProfileObj
        Debug.Print(" Tool body = " & swProfileBody.Name)
        swPathFeature = swSweepFeatureData.Path
        Debug.Print(" Path = " & swPathFeature.Name)

        ' Roll forward
        swSweepFeatureData.ReleaseSelectionAccess()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Cut Extrude in Sheet Metal Part Example (C#)

This example shows how to create an optimized, normal, cut extrude in a sheet metal part.

```
//-----
// Preconditions:
// 1. Verify that the sheet metal part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the sheet metal part.
// 2. Selects a face.
// 3. Sketches a circle on the selected face.
// 4. Creates Cut-Extrude2, which is an optimized, normal, cut
//    extrude.
// 5. Examine the FeatureManager design tree, graphics area, and
//    Immediate window.
//
// NOTE: Because the part document is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            ExtrudeFeatureData2 swExtrudeFeatureData = default(ExtrudeFeatureData2);
            bool status = false;
            int errors = 0;
            int warnings = 0;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-
sm.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Select a face
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByRay(-0.0287275955400048, 0.0301558252805876, 0.00509460972091347, 0.369531106281609, -0.495005022745071, -0.786
394804756136, 0.00115698538052806, 2, false, 0,
0);

            //Sketch a circle
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.004122, -0.003029, 0.0);

            //Create an optimized, normal, cut extrude
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureCut4(true, false, false, 1, 0, 0.01, 0.01, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
true, true, false, 0, 0, false, true);

            swModel.ClearSelection2(true);

            swExtrudeFeatureData = (ExtrudeFeatureData2)swFeature.GetDefinition();
            Debug.Print("Normal cut? " + swExtrudeFeatureData.NormalCut);
            Debug.Print("Geometry optimized? " + swExtrudeFeatureData.OptimizeGeometry);

        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Cut Extrude in Sheet Metal Part Example (VBA)

This example shows how to create an optimized, normal, cut extrude in a sheet metal part.

```
'-----
' Preconditions:
' 1. Verify that the sheet metal part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the sheet metal part.
' 2. Selects a face.
' 3. Sketches a circle on the selected face.
' 4. Creates Cut-Extrude2, which is an optimized, normal, cut
'    extrude.
' 5. Examine the FeatureManager design tree, graphics area,
'    Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not save
' changes.
'-----
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swExtrudeFeatureData as SldWorks.ExtrudeFeatureData2
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt", swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select a face
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByRay(-2.87275955400048E-02, 3.01558252805876E-02, 5.09460972091347E-03, 0.369531106281609, -0.495005022745071,
-0.786394804756136, 1.15698538052806E-03, 2, False, 0, 0)

    'Sketch a circle
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.004122, -0.003029, 0#)

    'Create an optimized, normal, cut extrude
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureCut4(True, False, False, 1, 0, 0.01, 0.01, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, True, True, True, True, False, 0, 0, False, True)

    swModel.ClearSelection2 True

    Set swExtrudeFeatureData = swFeature.GetDefinition
    Debug.Print "Normal cut? " & swExtrudeFeatureData.NormalCut
    Debug.Print "Geometry optimized? " & swExtrudeFeatureData.OptimizeGeometry

End Sub
```

Create Cut Extrude in Sheet Metal Part Example (VB.NET)

This example shows how to create an optimized, normal, cut extrude in a sheet metal part.

```

-----
Preconditions:
1. Verify that the sheet metal part to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the sheet metal part.
2. Selects a face.
3. Sketches a circle on the selected face.
4. Creates Cut-Extrude2, which is an optimized, normal, cut extrude.
5. Examine the FeatureManager design tree, graphics area, and Immediate window.

NOTE: Because the part document is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swExtrudeFeatureData As ExtrudeFeatureData2
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select a face
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByRay(-0.0287275955400048, 0.0301558252805876, 0.00509460972091347, 0.369531106281609, -0.495005022745071, -0.786394804756136, 0.00115698538052806, 2, False, 0, 0)

        'Sketch a circle
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.004122, -0.003029, 0.0#)

        'Create an optimized, normal, cut extrude
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureCut4(True, False, False, 1, 0, 0.01, 0.01, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, True, True, True, True, False, 0, 0, False, True)

        swModel.ClearSelection2(True)

        swExtrudeFeatureData = swFeature.GetDefinition
        Debug.Print("Normal cut? " & swExtrudeFeatureData.NormalCut)
        Debug.Print("Geometry optimized? " & swExtrudeFeatureData.OptimizeGeometry)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Cylinder Example (C++ COM)

This example shows how to create a cylinder.

```
'-----
'
' Preconditions: None.
'
' Postconditions: A cylinder is created.
'
'-----

#include "stdafx.h"
#include "CreateCylinder.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

#include <atlbase.h>

#import "sldworks.tlb" no_namespace raw_interfaces_only
#import "swconst.tlb" no_namespace

////////////////////////////////////
// The one and only application object
CWinApp theApp;

using namespace std;

int _tmain(int argc, TCHAR* argv[], TCHAR* envp[])
{
    int nRetCode = 0;
    // initialize MFC and print and error on failure
    if (!AfxWinInit(::GetModuleHandle(NULL), NULL, ::GetCommandLine(), 0))
    {
        // TODO: change error code to suit your needs
        cerr << _T("Fatal Error: MFC initialization failed") << endl;
        nRetCode = 1;
    }
    else
    {
        ::CoInitialize(NULL);
        {
            VARIANT_BOOL bRetVal = VARIANT_FALSE;

            CComPtr<ISldWorks> swApp;
            swApp.CoCreateInstance(L"SldWorks.Application", NULL, CLSCTX_LOCAL_SERVER);

            CComPtr<IModelDoc2> swDoc;
            CComPtr<IModelDocExtension> swDocExt;
            CComPtr<IFeatureManager> swFeatMgr;

            swApp->get_IActiveDoc2(&swDoc);
            swDoc->get_Extension(&swDocExt);
            swDoc->get_FeatureManager(&swFeatMgr);

            CComPtr<ISketch> swSketch;
            swDoc->IGetActiveSketch2(&swSketch);
            if (swSketch == NULL)
            {
                swDocExt->SelectByID2(L"Front Plane", L"PLANE", 0.0, 0.0, 0.0, VARIANT_FALSE, 0, NULL, swSelectOptionDefault, &bRetVal);
                swDoc->InsertSketch2(VARIANT_TRUE);
            }

            CComPtr<ISketchSegment> swSkSeg;
            swDoc->ICreateCircle2(0.0, 0.0, 0.0, 0.025, 0.0, 0.0, &swSkSeg);
```

```
        CComPtr<IFeature> swFeat;  
        swFeatMgr->FeatureExtrusion2(VARIANT_FALSE, VARIANT_FALSE, VARIANT_FALSE,  
            0, 0, 0.100, 0.100,  
            VARIANT_FALSE, VARIANT_FALSE, VARIANT_FALSE, VARIANT_FALSE, 0.0, 0.0,  
            VARIANT_FALSE, VARIANT_FALSE, VARIANT_FALSE, VARIANT_FALSE,  
            VARIANT_TRUE, VARIANT_FALSE, VARIANT_TRUE, 0, 0.0, VARIANT_FALSE, &swFeat);  
    }  
    ::CoUninitialize();  
}  
return nRetCode;  
}
```

Create Derived or Underived Sketch Example (C#)

This example shows how to create a derived or underived sketch.

```
//-----
// Preconditions:
// 1. Open a part that contains at least one sketch.
// 2. Select a sketch.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. If the selected sketch is not derived, then a
//    derived sketch is created.
//    - or -
//    If the selected sketch is derived, then the
//    sketch is changed to underived.
// 2. Examine the FeatureManager design tree and Immediate
//    window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace DeriveSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            Sketch swSketch = default(Sketch);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Interactively select a sketch
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSketch = (Sketch)swFeat.GetSpecificFeature2();

            // Is the selected sketch derived?
            Debug.Print("Is the selected sketch derived? " + swSketch.IsDerived());
            // If the selected sketch is a derived sketch,
            // then create a derived sketch; else, underive the
            // selected sketch
            if (swSketch.IsDerived())
            {
                swModel.UnderiveSketch();
                Debug.Print(" Selected sketch was derived; sketch is now underived.");
            }
            else
            {
                swModel.DeriveSketch();
                Debug.Print(" Selected sketch was not derived; a derived sketch has been created.");
            }
            swModel.ForceRebuild3(false);
        }
        public SldWorks swApp;
    }
}
```

Create Derived or Underived Sketch Example (VBA)

This example shows how to create a derived or underived sketch.

```
'-----
' Preconditions:
' 1. Open a part that contains at least one sketch.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. If the selected sketch is not derived, then a
'    derived sketch is created.
'    - or -
'    If the selected sketch is derived, then the
'    sketch is changed to underived.
' 2. Examine the FeatureManager design tree and Immediate
'    window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Interactively select a sketch
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swFeat.GetSpecificFeature2

    ' Is the selected sketch derived?
    Debug.Print ("Is the selected sketch derived? " & swSketch.IsDerived)

    ' If the selected sketch is a derived sketch,
    ' then create a derived sketch; else underive the
    ' selected sketch
    If swSketch.IsDerived Then
        swModel.UnderiveSketch
        Debug.Print (" Selected sketch was derived; sketch is now underived.")
    Else
        swModel.DeriveSketch
        Debug.Print (" Selected sketch was not derived; a derived sketch has been created.")
    End If

    swModel.ForceRebuild3 (False)

End Sub
```

Create Derived or Underived Sketch Example (VB.NET)

This example shows how to create a derived or underived sketch.

```

'-----
' Preconditions:
' 1. Open a part that contains at least one sketch.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. If the selected sketch is not derived, then a
'    derived sketch is created.
'    - or -
'    If the selected sketch is derived, then the
'    sketch is changed to underived.
' 2. Examine the FeatureManager design tree and Immediate
'    window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System
Imports System.Runtime.InteropServices

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swSketch As Sketch

        swModel = swApp.ActiveDoc

        ' Interactively select a sketch
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swSketch = swFeat.GetSpecificFeature2

        Debug.Print("Is the selected sketch derived? " & swSketch.IsDerived)

        ' If the selected sketch is a derived sketch,
        ' then create a derived sketch; else, underive the
        ' selected sketch
        If swSketch.IsDerived Then
            swModel.UnderiveSketch
            Debug.Print (" Selected sketch was derived; sketch is now underived.")
        Else
            swModel.DeriveSketch
            Debug.Print (" Selected sketch was not derived; a derived sketch has been created.")
        End If

        swModel.ForceRebuild3(False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Create Derived Pattern Feature Example (C#)

This example shows how to create a derived pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Creates a derived pattern feature.
// 3. Gets the type and name of the pattern feature used for
// the derived pattern feature.
// 4. Changes the position of the pattern instance used as the seed
// feature in the derived pattern feature.
// 5. Examine the FeatureManager design tree, Immediate window, and
// graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            DerivedPatternFeatureData swDerivedPatternFeatureData = default(DerivedPatternFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            Feature patternObj = default(Feature);

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\assem2.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExtension = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swModel.ClearSelection2(true);

            // Create derived pattern feature
            status = swModelDocExtension.SelectByID2("Part2\\assem2-7@assem2", "COMPONENT", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExtension.SelectByID2("LPattern1@Part3\\assem2-1@assem2", "BODYFEATURE", 0, 0, 0, true, 2, null, 0);
            swDerivedPatternFeatureData = (DerivedPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmDerivedLPattern);
            swFeature = (Feature)swFeatureManager.CreateFeature(swDerivedPatternFeatureData);

            // Get derived pattern feature name and type
            if ((swFeature != null))
            {
                Debug.Print("Derived pattern feature name = " + swFeature.Name);
                Debug.Print("Type of feature: " + swFeature.GetTypeName2());
                swDerivedPatternFeatureData = (DerivedPatternFeatureData)swFeature.GetDefinition();
                if ((swDerivedPatternFeatureData != null))
                {
                    swDerivedPatternFeatureData.AccessSelections(swModel, null);
                    // Get type of pattern feature used for derived pattern feature
                    patternObj = (Feature)swDerivedPatternFeatureData.PatternFeature;
                    Debug.Print("Type of pattern feature used for derived pattern feature: " + patternObj.GetTypeName2());
                    // Get pattern feature name
                    Debug.Print("Name of pattern feature used for the derived pattern feature: " + patternObj.Name);
                    LinearPatternFeatureData swLinearPatternFeatureData = default(LinearPatternFeatureData);
                    // Get the pattern feature used for the derived pattern feature
                    swLinearPatternFeatureData = (LinearPatternFeatureData)patternObj.GetDefinition();
                    int nbrPatternInstances = 0;
                    nbrPatternInstances = (swLinearPatternFeatureData.D1TotalInstances * swLinearPatternFeatureData.D2TotalInstances);
                    Debug.Print("Total number of pattern instances in pattern feature used for the derived pattern feature: " + nbrPatternInstances);
                    // Get position of the pattern instance used as the seed feature in the derived pattern feature
                    Debug.Print("Position of the pattern instance used as the seed feature in the derived pattern feature: " + swDerivedPatternFeatureData.SeedPosition);
                    // Change position of the pattern instance to use as the seed feature in the derived pattern feature
                    swDerivedPatternFeatureData.SeedPosition = nbrPatternInstances - 1;
                    Debug.Print("Modified position of the pattern instance to use as the seed feature in the derived pattern feature: " + swDerivedPatternFeatureData.SeedPosition);
                    swFeature.ModifyDefinition(swDerivedPatternFeatureData, swModel, null);
                }
            }

            swModel.ClearSelection2(true);
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Create Derived Pattern Feature Example (VBA)

This example shows how to create a derived pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates a derived pattern feature.
' 3. Gets the type and name of the pattern feature used for
' the derived pattern feature.
' 4. Changes the position of the pattern instance used as the seed
' feature in the derived pattern feature.
' 5. Examine the FeatureManager design tree, Immediate window, and
' graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExtension As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swDerivedPatternFeatureData As SldWorks.DerivedPatternFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim patternObj As SldWorks.Feature

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem2.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExtension = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager

    swModel.ClearSelection2 True

    ' Create derived pattern feature
    status = swModelDocExtension.SelectByID2("Part2^assem2-7@assem2", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExtension.SelectByID2("LPattern1@Part3^assem2-1@assem2", "BODYFEATURE", 0, 0, 0, True, 2, Nothing, 0)

    Set swDerivedPatternFeatureData = swFeatureManager.CreateDefinition(swFmDerivedLPattern)
    Set swFeature = swFeatureManager.CreateFeature(swDerivedPatternFeatureData)

    ' Get derived pattern feature name and type
    If Not swFeature Is Nothing Then
        Debug.Print "Derived pattern feature name = " & swFeature.Name
        Debug.Print " Type of feature: " & swFeature.GetTypeName2
        Set swDerivedPatternFeatureData = swFeature.GetDefinition()
        If Not swDerivedPatternFeatureData Is Nothing Then
            swDerivedPatternFeatureData.AccessSelections swModel, Nothing
            ' Get type of pattern feature used for derived pattern feature
            Set patternObj = swDerivedPatternFeatureData.PatternFeature
            Debug.Print " Type of pattern feature used for derived pattern feature: " & patternObj.GetTypeName2
            ' Get pattern feature name
            Debug.Print " Name of pattern feature used for the derived pattern feature: " & patternObj.Name
            Dim swLinearPatternFeatureData As SldWorks.LinearPatternFeatureData
            ' Get the pattern feature used for the derived pattern feature
            Set swLinearPatternFeatureData = patternObj.GetDefinition
            Dim nbrPatternInstances As Long
            nbrPatternInstances = (swLinearPatternFeatureData.D1TotalInstances * swLinearPatternFeatureData.D2TotalInstances)
            Debug.Print " Total number of pattern instances in pattern feature used for the derived pattern feature: " & nbrPatternInstances
            ' Get position of the pattern instance used as the seed feature in the derived pattern feature
            Debug.Print " Position of the pattern instance used as the seed feature in the derived pattern feature: " &
            swDerivedPatternFeatureData.SeedPosition
            ' Change position of the pattern instance to use as the seed feature in the derived pattern feature
            swDerivedPatternFeatureData.SeedPosition = nbrPatternInstances - 1
            Debug.Print " Modified position of the pattern instance to use as the seed feature in the derived pattern feature: " &
            swDerivedPatternFeatureData.SeedPosition
            swFeature.ModifyDefinition swDerivedPatternFeatureData, swModel, Nothing
        End If
    End If

    swModel.ClearSelection2 True

End Sub

```

Create Derived Pattern Feature Example (VB.NET)

This example shows how to create a derived pattern feature.

```

-----
Preconditions:
1. Verify that the assembly to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the assembly.
2. Creates a derived pattern feature.
3. Gets the type and name of the pattern feature used for
the derived pattern feature.
4. Changes the position of the pattern instance used as the seed
feature in the derived pattern feature.
5. Examine the FeatureManager design tree, Immediate window, and
graphics area.

NOTE: Because the assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swDerivedPatternFeatureData As DerivedPatternFeatureData
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim patternObj As Feature

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2019\samples\tutorial\api\assem2.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExtension = swModel.Extension
        swFeatureManager = swModel.FeatureManager

        swModel.ClearSelection2(True)

        ' Create derived pattern feature
        status = swModelDocExtension.SelectByID2("Part2^assem2-7@assem2", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExtension.SelectByID2("LPattern1@Part3^assem2-1@assem2", "BODYFEATURE", 0, 0, 0, True, 2, Nothing, 0)

        swDerivedPatternFeatureData = swFeatureManager.CreateDefinition(swFeatureNameID_e.swFmDerivedLPattern)
        swFeature = swFeatureManager.CreateFeature(swDerivedPatternFeatureData)

        ' Get derived pattern feature name and type
        If Not swFeature Is Nothing Then
            Debug.Print("Derived pattern feature name = " & swFeature.Name)
            Debug.Print(" Type of feature: " & swFeature.GetTypeName2)
            swDerivedPatternFeatureData = swFeature.GetDefinition()
            If Not swDerivedPatternFeatureData Is Nothing Then
                swDerivedPatternFeatureData.AccessSelections(swModel, Nothing)
                ' Get type of pattern feature used for derived pattern feature
                patternObj = swDerivedPatternFeatureData.PatternFeature
                Debug.Print(" Type of pattern feature used for derived pattern feature: " & patternObj.GetTypeName2)
                ' Get pattern feature name
                Debug.Print(" Name of pattern feature used for the derived pattern feature: " & patternObj.Name)
                Dim swLinearPatternFeatureData As LinearPatternFeatureData
                ' Get the pattern feature used for the derived pattern feature
                swLinearPatternFeatureData = patternObj.GetDefinition
                Dim nbrPatternInstances As Integer
                nbrPatternInstances = (swLinearPatternFeatureData.D1TotalInstances * swLinearPatternFeatureData.D2TotalInstances)
                Debug.Print(" Total number of pattern instances in pattern feature used for the derived pattern feature: " & nbrPatternInstances)
                ' Get position of the pattern instance used as the seed feature in the derived pattern feature
                Debug.Print(" Position of the pattern instance used as the seed feature in the derived pattern feature: " & swDerivedPatternFeatureData.SeedPosition)
                ' Change position of the pattern instance to use as the seed feature in the derived pattern feature
                swDerivedPatternFeatureData.SeedPosition = nbrPatternInstances - 1

                Debug.Print(" Modified position of the pattern instance to use as the seed feature in the derived pattern feature: " & swDerivedPatternFeatureData.SeedPosition)
                swFeature.ModifyDefinition(swDerivedPatternFeatureData, swModel, Nothing)
            End If
        End If

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Detail Circle and Detail View Example (C#)

This example shows how to create a detail circle and a detail view.

```
//-----
// Preconditions:
// 1. Verify that the drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Activates Drawing View4.
// 3. Creates a detail circle and a detail view using the visible
//    corner of Drawing View4.
// 4. Activates the detail view.
// 5. Gets and sets some properties of the detail circle and detail view.
// 6. Examine the drawing document and Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            View swView = default(View);
            DetailCircle swDetailCircle = default(DetailCircle);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            // Open drawing
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\replaceview.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swApp.ActivateDoc3("replaceview - Sheet1", false, (int)swRebuildOnActivation_e.swDontRebuildActiveDoc, ref errors);

            // Activate Drawing View4 and create detail circle and detail view
            status = swDrawing.ActivateView("Drawing View4");
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.007581, 0.053509, 0.0, 0.013533, 0.016475, 0.0);
            swView = (View)swDrawing.CreateDetailViewAt4(0.22305342706156, 0.0762140266484527, 0, (int)swDetViewStyle_e.swDetViewSTANDARD, 1, 1, "A", (int)swDetCircleShowType_e
true, false, 5);

            swModel.ClearSelection2(true);

            // Activate detail view
            status = swDrawing.ActivateView("Drawing View5");

            // Get and set some properties of detail circle and detail view
            swDetailCircle = (DetailCircle)swView.GetDetail();
            Debug.Print("Detail circle:");
            Debug.Print(" Selected: " + swDetailCircle.Select(true, null));
            Debug.Print(" Label: " + swDetailCircle.GetLabel());
            Double xpos;
            Double ypos;
            swDetailCircle.GetLabelPosition(out xpos, out ypos);
            Debug.Print(" Label X position: " + xpos);
            Debug.Print(" Label Y position: " + ypos);
            Debug.Print(" Type of circle: " + swDetailCircle.GetDisplay());
            Debug.Print(" Name: " + swDetailCircle.GetName());
            Debug.Print(" Style: " + swDetailCircle.GetStyle());
            Debug.Print(" Default document text formatting? " + swDetailCircle.GetUseDocTextFormat());
            if (swDetailCircle.NoOutline == false)
            {
                Debug.Print(" No outline? False");
                if (swDetailCircle.JaggedOutline == true)
```

```
        {
            swDetailCircle.ShapeIntensity = 2;
            Debug.Print(" Jagged outline and shape intensity? True and 2");
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}
```

Create Detail Circle and Detail View Example (VBA)

This example shows how to create a detail circle and a detail view.

```

'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Activates Drawing View4.
' 3. Creates a detail circle and a detail view using the visible
'    corner of Drawing View4.
' 4. Activates the detail view.
' 5. Gets and sets some properties of the detail circle and detail view.
' 6. Examine the drawing document and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swView As SldWorks.View
Dim swDetailCircle As SldWorks.DetailCircle
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim fileName As String
Dim status As Boolean
Dim errors As Long, warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open drawing
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swDrawing = swModel
    swApp.ActivateDoc3 "replaceview - Sheet1", False, swDontRebuildActiveDoc, errors

    ' Activate Drawing View4 and create detail circle and detail view
    status = swDrawing.ActivateView("Drawing View4")
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateCircle(0.007581, 0.053509, 0#, 0.013533, 0.016475, 0#)
    Set swView = swDrawing.CreateDetailViewAt4(0.22305342706156, 7.62140266484527E-02, 0, swDetViewSTANDARD, 1, 1, "A", swDetCircleCIRCLE, True, True,
    False, 5)

    swModel.ClearSelection2 True

    ' Activate detail view
    status = swDrawing.ActivateView("Drawing View5")

    ' Get and set some properties of detail circle and detail view
    Set swDetailCircle = swView.GetDetail
    Debug.Print "Detail circle:"
    Debug.Print "    Selected: " & swDetailCircle.Select(True, Nothing)
    Debug.Print "    Label: " & swDetailCircle.GetLabel
    Dim xpos As Double
    Dim ypos As Double
    swDetailCircle.GetLabelPosition xpos, ypos
    Debug.Print "    Label X position: " & xpos
    Debug.Print "    Label Y position: " & ypos
    Debug.Print "    Type of circle: " & swDetailCircle.GetDisplay
    Debug.Print "    Name: " & swDetailCircle.GetName
    Debug.Print "    Style: " & swDetailCircle.GetStyle
    Debug.Print "    Default document text formatting? " & swDetailCircle.GetUseDocTextFormat
    If Not swDetailCircle.NoOutline Then
        Debug.Print "    No outline? False"
        If swDetailCircle.JaggedOutline Then
            swDetailCircle.ShapeIntensity = 2
            Debug.Print "    Jagged outline and shape intensity? True and 2"
        End If
    End If
End Sub

```

Create Detail Circle and Detail View Example (VB.NET)

This example shows how to create a detail circle and a detail view.

```

'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Activates Drawing View4.
' 3. Creates a detail circle and a detail view using the visible
'    corner of Drawing View4.
' 4. Activates the detail view.
' 5. Gets and sets some properties of the detail circle and detail view.
' 6. Examine the drawing document and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swView As View
        Dim swDetailCircle As DetailCircle
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer

        ' Open drawing
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\replaceview.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, v
        swDrawing = swModel
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swApp.ActivateDoc3("replaceview - Sheet1", False, swRebuildOnActivation_e.swDontRebuildActiveDoc, errors)

        ' Activate Drawing View4 and create detail circle and detail view
        status = swDrawing.ActivateView("Drawing View4")
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateCircle(0.007581, 0.053509, 0.0#, 0.013533, 0.016475, 0.0#)
        swView = swDrawing.CreateDetailViewAt4(0.22305342706156, 0.0762140266484527, 0, swDetViewStyle_e.swDetViewSTANDARD, 1, 1, "#
True, False, 5)

        swModel.ClearSelection2(True)

        ' Activate detail view
        status = swDrawing.ActivateView("Drawing View5")

        ' Get and some properties of detail circle and detail view
        swDetailCircle = swView.GetDetail
        Debug.Print("Detail circle:")
        Debug.Print("  Selected: " & swDetailCircle.Select(True, Nothing))
        Debug.Print("  Label: " & swDetailCircle.GetLabel)
        Dim xpos As Double
        Dim ypos As Double
        swDetailCircle.GetLabelPosition(xpos, ypos)
        Debug.Print("  Label X position: " & xpos)
        Debug.Print("  Label Y position: " & ypos)
        Debug.Print("  Type of circle: " & swDetailCircle.GetDisplay)
        Debug.Print("  Name: " & swDetailCircle.GetName)
        Debug.Print("  Style: " & swDetailCircle.GetStyle)
        Debug.Print("  Default document text formatting? " & swDetailCircle.GetUseDocTextFormat)
        If (swDetailCircle.NoOutline = Then
            Debug.Print("  No outline? False")
            If swDetailCircle.JaggedOutline Then
                swDetailCircle.ShapeIntensity = 2
                Debug.Print("  Jagged outline and shape intensity? True and 2")
            End If
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Create Detail View Example (VBA)

This example shows how to create a detail view.

A named view is activated and a circle centered about the view's origin is created. A detail view is then created at the origin of the sheet. If you modify the macro to adjust the values to different locations, you can see how the values affect the results.

The 1,1,0 represent 1m,1m,0m in sheet space. Most APIs expect units of meters and radians for length and angle values, respectively.

NOTE: The call to IDrawingDoc::CreateDetailViewAt3 uses the x,y,z coordinates relative to the sheet's origin. The sheet's origin is at the lower-left corner of the sheet. The x,y,z coordinates represent the location on the sheet where to place the origin of the parent view.

```
'-----  
'  
' Preconditions: Drawing document is open and contains a Drawing View1.  
'  
' Postconditions: A circle is created about the activated view's origin  
'               and a detail view is then created at the origin of the sheet.  
'  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawingDoc As SldWorks.DrawingDoc  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swSketchSegment As SldWorks.SketchSegment
```

Sub main()

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swDrawingDoc = swModel
```

```
swDrawingDoc.ActivateView "Drawing View1"  
Set swSketchSegment = swModel.CreateCircle2(0, 0, 0, 0.005, 0, 0)  
swDrawingDoc.CreateDetailViewAt3 0, 0, 0, 0, 1#, "B", 1, 0
```

End Sub

Create Drag Arrow Manipulator Example (C#)

This example shows how to create a drag arrow manipulator, which is called a handle in the user interface.

```
//-----
// Preconditions:
// 1. Open a part or assembly and select a face.
// 2. De-select Tools > Options > Stop VSTA debugger on macro exit.
// 3. Select Project > Add Reference > .NET > SolidWorks.Interop.swpublished.
// 4. Rename the namespace to match the name of your C# project.
// 5. Open an Immediate window.
//
// Postconditions:
// 1. A unidirectional drag arrow manipulator or handle is created on the selected face.
// 2. Drag the handle to another location. The first time you drag the handle,
//    IDragArrowManipulator::FixedLength is set to true,
//    and the origin is moved in the direction of the drag. For second and
//    subsequent drags, IDragArrowManipulator::FixedLength is set to false,
//    and the origin is not changed.
// 3. When you drag the handle a ruler displays.
//    IDragArrowManipulator::ShowRuler is true.
// 4. When you drag the handle past length = 0, the handle flips direction.
//    IDragArrowManipulator::AllowFlip is true.
// 5. Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime;
using System.Diagnostics;
namespace CreateDragArrow_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        Manipulator swManip;
        DragArrowManipulator swDrag;
        SwManHandler2 swDragHdlr = new SwManHandler2();

        public void Main()
        {
            Face2 swFace;
            MathUtility swMathUtil = default(MathUtility);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            object vPickPt = null;
            MathPoint swPickPt = default(MathPoint);

            swMathUtil = (MathUtility)swApp.GetMathUtility();
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, 0);

            double[] nVector = new double[3];
            object vVector = null;
            nVector[0] = 0;
            nVector[1] = 1;
            nVector[2] = 0;
            vVector = nVector;

            MathVector swN = default(MathVector);
            swN = (MathVector)swMathUtil.CreateVector((vVector));

            vPickPt = swSelMgr.GetSelectionPoint2(1, -1);
            swPickPt = (MathPoint)swMathUtil.CreatePoint((vPickPt));
            swModViewMgr = swModel.ModelViewManager;

            swManip = swModViewMgr.CreateManipulator(((int)swManipulatorType_e.swDragArrowManipulator, swDragHdlr);
            swDrag = (DragArrowManipulator)swManip.GetSpecificManipulator();

            swDrag.AllowFlip = true;
            swDrag.ShowRuler = true;
            swDrag.ShowOppositeDirection = true;
            swDrag.Length = 0.02;
            swDrag.Direction = swN;
            swDrag.LengthOppositeDirection = 0.01;
            swDrag.Origin = swPickPt;
            swManip.Show(swModel);
            swDrag.Update();

            MathPoint origin = default(MathPoint);
            origin = swDrag.Origin;

            object pt = null;
            pt = origin.ArrayData;
        }
    }
}
```

```

    public SldWorks swApp;
}
[System.Runtime.InteropServices.ComVisible(true)]
public class SwManHandler2 : SwManipulatorHandler2
{
    int doneonce;

    const int lenFact = 1;
    private bool SwManipulatorHandler2_OnHandleLmbSelected(object pManipulator)
    {
        Debug.Print("SwManipulatorHandler2_OnHandleLmbSelected");
        return true;
    }
    bool ISwManipulatorHandler2.OnHandleLmbSelected(object pManipulator)
    {
        return SwManipulatorHandler2_OnHandleLmbSelected(pManipulator);
    }
    private bool SwManipulatorHandler2_OnDelete(object pManipulator)
    {
        Debug.Print("SwManipulatorHandler2_OnDelete");
        return true;
    }
    bool ISwManipulatorHandler2.OnDelete(object pManipulator)
    {
        return SwManipulatorHandler2_OnDelete(pManipulator);
    }
    private void SwManipulatorHandler2_OnDirectionFlipped(object pManipulator)
    {
        Debug.Print("SwManipulatorHandler2_OnDirectionFlipped");
    }
    void ISwManipulatorHandler2.OnDirectionFlipped(object pManipulator)
    {
        SwManipulatorHandler2_OnDirectionFlipped(pManipulator);
    }

    private bool SwManipulatorHandler2_OnDoubleValueChanged(object pManipulator, int Id, ref double Value)
    {
        doneonce = doneonce + 1;
        Debug.Print("SwManipulatorHandler2_OnDoubleValueChanged");

        Debug.Print(" ID          = " + Id);

        Debug.Print(" Value          = " + Value);
        DragArrowManipulator swTmpManipulator = default(DragArrowManipulator);
        swTmpManipulator = (DragArrowManipulator)pManipulator;
        //Update origin
        MathPoint swMathPoint = default(MathPoint);
        swMathPoint = swTmpManipulator.Origin;
        double[] varMathPt = null;
        varMathPt = (double[])swMathPoint.ArrayData;
        varMathPt[1] = varMathPt[1] + lenFact / 1000;
        swMathPoint.ArrayData = varMathPt;
        if ((doneonce == 1))
        {
            swTmpManipulator.FixedLength = true;
        }
        swTmpManipulator.Origin = swMathPoint;

        swTmpManipulator.Update();
        return true;
    }
    bool ISwManipulatorHandler2.OnDoubleValueChanged(object pManipulator, int Id, ref double Value)
    {
        return SwManipulatorHandler2_OnDoubleValueChanged(pManipulator, Id, ref Value);
    }

    private void SwManipulatorHandler2_OnEndDrag(object pManipulator, int handleIndex)
    {
        DragArrowManipulator swTmpManipulator = default(DragArrowManipulator);
        swTmpManipulator = (DragArrowManipulator)pManipulator;
        swTmpManipulator.FixedLength = false;
        doneonce = doneonce + 1;
        Debug.Print("SwManipulatorHandler2_OnEndDrag");

        Debug.Print(" HandleIndex      = " + handleIndex);

        if ((handleIndex == (int)swDragArrowManipulatorOptions_e.swDragArrowManipulatorDirection2))
        {
            Debug.Print(" Direction1");
        }
    }
}

```

```

    }
    else
    {
        Debug.Print(" Direction2");
    }
}

void ISwManipulatorHandler2.OnEndDrag(object pManipulator, int handleIndex)
{
    SwManipulatorHandler2_OnEndDrag(pManipulator, handleIndex);
}

private void SwManipulatorHandler2_OnEndNoDrag(object pManipulator, int handleIndex)
{
    Debug.Print("SwManipulatorHandler2_OnEndNoDrag");
}

void ISwManipulatorHandler2.OnEndNoDrag(object pManipulator, int handleIndex)
{
    SwManipulatorHandler2_OnEndNoDrag(pManipulator, handleIndex);
}

private void SwManipulatorHandler2_OnHandleRmbSelected(object pManipulator, int handleIndex)
{
    Debug.Print("SwManipulatorHandler2_OnHandleRmbSelected");
    Debug.Print(" handleIndex      = " + handleIndex);
}

void ISwManipulatorHandler2.OnHandleRmbSelected(object pManipulator, int handleIndex)
{
    SwManipulatorHandler2_OnHandleRmbSelected(pManipulator, handleIndex);
}

private void SwManipulatorHandler2_OnHandleSelected(object pManipulator, int handleIndex)
{
    Debug.Print("SwManipulatorHandler2_OnHandleSelected");
    Debug.Print(" HandleIndex      = " + handleIndex);
}

void ISwManipulatorHandler2.OnHandleSelected(object pManipulator, int handleIndex)
{
    SwManipulatorHandler2_OnHandleSelected(pManipulator, handleIndex);
}

private void SwManipulatorHandler2_OnItemSetFocus(object pManipulator, int Id)
{
    Debug.Print("SwManipulatorHandler2_OnItemSetFocus");
    Debug.Print(" ID              = " + Id);
}

void ISwManipulatorHandler2.OnItemSetFocus(object pManipulator, int Id)
{
    SwManipulatorHandler2_OnItemSetFocus(pManipulator, Id);
}

private bool SwManipulatorHandler2_OnStringValueChanged(object pManipulator, int Id, ref string Value)
{
    Debug.Print("SwManipulatorHandler2_OnStringValueChanged");
    Debug.Print(" ID              = " + Id);
    Debug.Print(" Value          = " + Value);
    return true;
}

bool ISwManipulatorHandler2.OnStringValueChanged(object pManipulator, int Id, ref string Value)
{
    return SwManipulatorHandler2_OnStringValueChanged(pManipulator, Id, ref Value);
}

private void SwManipulatorHandler2_OnUpdateDrag(object pManipulator, int handleIndex, object newPosMathPt)
{
    Debug.Print("SwManipulatorHandler2_OnUpdateDrag");
    Debug.Print(" HandleIndex    = " + handleIndex);
}

void ISwManipulatorHandler2.OnUpdateDrag(object pManipulator, int handleIndex, object newPosMathPt)

```

```
    {
        SwManipulatorHandler2_OnUpdateDrag(pManipulator, handleIndex, newPosMathPt);
    }
}
```

Create Drag Arrow Manipulator Example (VBA)

This example shows how to create a drag arrow manipulator, which is called a handle in the user interface.

- [Main Module](#)
- [Handler Module](#)

Main module

```

'-----
' Preconditions:
' 1. Open a part or assembly and select a face.
' 2. Select:
'     Tools > References > SOLIDWORKS <version> exposed type libraries for add-in
' 3. Open an Immediate window.
'
' Postconditions:
' 1. A unidirectional drag arrow manipulator or handle is created on the selected face.
' 2. Drag the handle to another location. The first time you drag the handle,
'     IDragArrowManipulator::FixedLength is set to true,
'     and the origin is moved in the direction of the drag. For second and
'     subsequent drags, IDragArrowManipulator::FixedLength is set to false,
'     and the origin is not changed.
' 3. When you drag the handle a ruler displays.
'     IDragArrowManipulator::ShowRuler is true.
' 4. When you drag the handle past length = 0, the handle flips direction.
'     IDragArrowManipulator::AllowFlip is true.
' 5. Inspect the Immediate window.
'-----

```

Option Explicit

```

Dim swManip           As SldWorks.Manipulator
Public swDrag          As SldWorks.DragArrowManipulator
Dim swDragHdlr        As swDragManipHdlr
Public swFace          As SldWorks.Face2

```

Sub main()

```

Dim swApp              As SldWorks.SldWorks
Dim swMathUtil         As SldWorks.MathUtility
Dim swModel            As SldWorks.ModelDoc2
Dim swModViewMgr       As SldWorks.ModelViewManager
Dim swSelMgr           As SldWorks.SelectionMgr
Dim vPickPt            As Variant
Dim swPickPt           As SldWorks.MathPoint
Dim boolStatus As Boolean

Set swDragHdlr = New swDragManipHdlr
Set swApp = Application.SldWorks
Set swMathUtil = swApp.GetMathUtility
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFace = swSelMgr.GetSelectedObject6(1, 0)

Dim nVector(2)         As Double
Dim vVector            As Variant
nVector(0) = 0:   nVector(1) = 1:   nVector(2) = 0
vVector = nVector

Dim swN                As SldWorks.MathVector
Set swN = swMathUtil.CreateVector((vVector))

vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
Set swPickPt = swMathUtil.CreatePoint((vPickPt))
Set swModViewMgr = swModel.ModelViewManager

Set swManip = swModViewMgr.CreateManipulator(SwConst.swManipulatorType_e.swDragArrowManipulator, swDragHdlr)
Set swDrag = swManip.GetSpecificManipulator

swDrag.ShowRuler = True
swDrag.AllowFlip = True
swDrag.ShowOppositeDirection = False
swDrag.Length = 0.02
swDrag.Direction = swN
swDrag.LengthOppositeDirection = 0.01
swDrag.origin = swPickPt
swManip.Show swModel
swDrag.Update

Dim origin As SldWorks.MathPoint
Set origin = swDrag.origin

Dim pt As Variant
pt = origin.ArrayData

```

End Sub

Handler module

```

Dim Doneonce As Long
'-----

```

```

Const lenFact = 1
Option Explicit
Implements SwManipulatorHandler2

Public Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean
    Debug.Print "SwManipulatorHandler2_OnHandleLmbSelected"
End Function

Public Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean
    Debug.Print "SwManipulatorHandler2_OnDelete"
End Function

Public Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object)
    Debug.Print "SwManipulatorHandler2_OnDirectionFlipped"
End Sub

Public Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As Double) As Boolean
    Doneonce = Doneonce + 1
    Debug.Print "SwManipulatorHandler2_OnDoubleValueChanged"

    Debug.Print " ID = " & Id
    Debug.Print " Value = " & Value
    Dim swTmpManipulator As SldWorks.DragArrowManipulator
    Set swTmpManipulator = pManipulator
    'Update origin
    Dim swMathPoint As SldWorks.MathPoint
    Set swMathPoint = swTmpManipulator.origin
    Dim varMathPt As Variant
    varMathPt = swMathPoint.ArrayData
    varMathPt(1) = varMathPt(1) + lenFact / 1000
    swMathPoint.ArrayData = varMathPt
    If (Doneonce = 1) Then
        swTmpManipulator.FixedLength = True
    End If
    swTmpManipulator.origin = swMathPoint

    swTmpManipulator.Update
End Function

Public Sub SwManipulatorHandler2_OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Dim swTmpManipulator As SldWorks.DragArrowManipulator
    Set swTmpManipulator = pManipulator
    swTmpManipulator.FixedLength = False
    Doneonce = Doneonce + 1
    Debug.Print "SwManipulatorHandler2_OnEndDrag"
    Debug.Print " HandleIndex = " & handleIndex

    If (handleIndex = swDragArrowManipulatorOptions_e.swDragArrowManipulatorDirection2) Then
        Debug.Print " Direction1"
    Else
        Debug.Print " Direction2"
    End If
End Sub

Public Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "SwManipulatorHandler2_OnEndNoDrag"
End Sub

Public Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "SwManipulatorHandler2_OnHandleRmbSelected"
    Debug.Print " handleIndex = " & handleIndex
End Sub

Public Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    Debug.Print "SwManipulatorHandler2_OnHandleSelected"
    Debug.Print " HandleIndex = " & handleIndex
End Sub

Public Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Long)
    Debug.Print "SwManipulatorHandler2_OnItemSetFocus"
    Debug.Print " ID = " & Id
End Sub

Public Function SwManipulatorHandler2_OnLmbSelected(ByVal pManipulator As Object) As Boolean
    Debug.Print "SwManipulatorHandler2_OnLmbSelected"
End Function

Public Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As String) As Boolean
    Debug.Print "SwManipulatorHandler2_OnStringValueChanged"
    Debug.Print " ID = " & Id
    Debug.Print " Value = " & Value

```

End Function

Public Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As Object)

 Debug.Print "SwManipulatorHandler2_OnUpdateDrag"

 Debug.Print " HandleIndex = " & handleIndex

End Sub

Create Drag Arrow Manipulator Example (VB.NET)

This example shows how to create a drag arrow manipulator, which is called a handle in the user interface.

```

'-----
' Preconditions:
' 1. Open a part or assembly and select a face.
' 2. Select Project > Add Reference > .NET > SolidWorks.Interop.swpublished.
' 3. De-select Tools > Options > Stop VSTA debugger on macro exit.
' 4. Copy the main class into SolidWorksMacro.vb of your VB.NET project.
' 5. Copy the handler class into Class1.vb of your VB.NET project.
' 6. Open an Immediate window.
'
' Postconditions:
' 1. A unidirectional drag arrow manipulator or handle is created on the selected face.
' 2. Drag the handle to another location. The first time you drag the handle,
'    IDragArrowManipulator::FixedLength is set to true,
'    and the origin is moved in the direction of the drag. For second and
'    subsequent drags, IDragArrowManipulator::FixedLength is set to false,
'    and the origin is not changed.
' 3. When you drag the handle a ruler displays.
'    IDragArrowManipulator::ShowRuler is true.
' 4. When you drag the handle past length = 0, the handle flips direction.
'    IDragArrowManipulator::AllowFlip is true.
' 5. Inspect the Immediate window.
'-----

```

'[main class](#):

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Public swManip As Manipulator
    Public swDrag As DragArrowManipulator
    Public WithEvents swDragHdlr As New Class1
    Public Sub Main()

        Dim swFace As Face2
        Dim swMathUtil As MathUtility
        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim swSelMgr As SelectionMgr
        Dim vPickPt As Object
        Dim swPickPt As MathPoint

        swMathUtil = swApp.GetMathUtility
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFace = swSelMgr.GetSelectedObject6(1, 0)

        Dim nVector(2) As Double
        Dim vVector As Object
        nVector(0) = 0 : nVector(1) = 1 : nVector(2) = 0
        vVector = nVector

        Dim swN As MathVector
        swN = swMathUtil.CreateVector((vVector))

        vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
        swPickPt = swMathUtil.CreatePoint((vPickPt))
        swModViewMgr = swModel.ModelViewManager

        swManip = swModViewMgr.CreateManipulator(1, swDragHdlr)
        swDrag = swManip.GetSpecificManipulator

        swDrag.AllowFlip = True
        swDrag.ShowRuler = True
        swDrag.ShowOppositeDirection = True
        swDrag.Length = 0.02
        swDrag.Direction = swN
        swDrag.LengthOppositeDirection = 0.01
        swDrag.Origin = swPickPt
        swManip.Show(swModel)
        swDrag.Update()

        Dim origin As MathPoint
        origin = swDrag.Origin

        Dim pt As Object
        pt = origin.ArrayData

    End Sub
    Public swApp As SldWorks
End Class

```

handler class:

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Diagnostics
Imports System.Runtime
<System.Runtime.InteropServices.ComVisibleAttribute(True)> _
Public Class Class1
    Implements SwManipulatorHandler2

    Dim doneonce As Integer
    Const lenFact As Integer = 1
    Private Function OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean Implements
ISwManipulatorHandler2.OnHandleLmbSelected

        Debug.Print("SwManipulatorHandler2_OnHandleLmbSelected")

    End Function
    Private Function OnDelete(ByVal pManipulator As Object) As Boolean Implements ISwManipulatorHandler2.OnDelete

        Debug.Print("SwManipulatorHandler2_OnDelete")

    End Function
    Private Sub OnDirectionFlipped(ByVal pManipulator As Object) Implements ISwManipulatorHandler2.OnDirectionFlipped

        Debug.Assert(False)

        Debug.Print("SwManipulatorHandler2_OnDirectionFlipped")

    End Sub
    Private Function OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Integer, ByRef Value As Double) As Boolean
Implements ISwManipulatorHandler2.OnDoubleValueChanged

        'Debug.Assert False
        doneonce = doneonce + 1
        Debug.Print("SwManipulatorHandler2_OnDoubleValueChanged")

        Debug.Print(" ID = " & Id)

        Debug.Print(" Value = " & Value)
        Dim swTmpManipulator As DragArrowManipulator
        swTmpManipulator = pManipulator
        'Update origin
        Dim swMathPoint As MathPoint
        swMathPoint = swTmpManipulator.Origin
        Dim varMathPt As Object
        varMathPt = swMathPoint.ArrayData
        varMathPt(1) = varMathPt(1) + lenFact / 1000
        swMathPoint.ArrayData = varMathPt
        If (doneonce = 1) Then
            swTmpManipulator.FixedLength = True
        End If
        swTmpManipulator.Origin = swMathPoint

        swTmpManipulator.Update()
        Return True
    End Function
    Private Sub OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements ISwManipulatorHandler2.OnEndDrag

        Dim swTmpManipulator As DragArrowManipulator
        swTmpManipulator = pManipulator
        swTmpManipulator.FixedLength = False
        doneonce = doneonce + 1
        Debug.Print("SwManipulatorHandler2_OnEndDrag")

        Debug.Print(" HandleIndex = " & handleIndex)

        If (handleIndex = swDragArrowManipulatorOptions_e.swDragArrowManipulatorDirection2) Then

            Debug.Print(" Direction1")

        Else

            Debug.Print(" Direction2")

        End If

    End Sub
    Private Sub OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
ISwManipulatorHandler2.OnEndNoDrag

        Debug.Print("SwManipulatorHandler2_OnEndNoDrag")

    End Sub
    Private Sub OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
ISwManipulatorHandler2.OnHandleRmbSelected

        Debug.Print("SwManipulatorHandler2_OnHandleRmbSelected")

        Debug.Print(" handleIndex = " + handleIndex)
```

```

End Sub
Private Sub OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
ISwManipulatorHandler2.OnHandleSelected

    Debug.Print("SwManipulatorHandler2_OnHandleSelected")

    Debug.Print("    HandleIndex        = " + handleIndex)

End Sub
Private Sub OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Integer) Implements ISwManipulatorHandler2.OnItemSetFocus

    Debug.Assert(False)

    Debug.Print("SwManipulatorHandler2_OnItemSetFocus")

    Debug.Print("    ID                = " & Id)

End Sub
Private Function OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Integer, ByRef Value As String) As Boolean
Implements ISwManipulatorHandler2.OnStringValueChanged

    Debug.Assert(False)

    Debug.Print("SwManipulatorHandler2_OnStringValueChanged")

    Debug.Print("    ID                = " & Id)

    Debug.Print("    Value            = " & Value)

End Function
Private Sub OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByVal newPosMathPt As Object) Implements
ISwManipulatorHandler2.OnUpdateDrag

    Debug.Print("SwManipulatorHandler2_OnUpdateDrag")

    Debug.Print("    HandleIndex      = " & handleIndex)

End Sub

Public Sub New()

End Sub
End Class

```

Create Drawing Sheet Zones Example (C#)

This example shows how to create a drawing sheet with zones, modify the zones in the drawing sheet, and insert a revision table.

```
//-----
// Preconditions:
// 1. Verify that the specified model document and templates exist.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new sheet named Test with four zones.
// 2. Inspect the graphics area.
// 3. Press F5.
// 4. Modifies Test to contain nine zones.
// 5. Creates Revision Table1.
// 6. Adds a revision row to the table.
// 7. Inspect the FeatureManager design tree, the graphics area, and the
//    Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes to it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertRevisionTable_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        DrawingDoc swDraw;
        Sheet currentsheet;
        ModelDoc2 swModel;
        RevisionTableAnnotation revTableAnno;
        bool boolstatus;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\assem20.slddrw", 3, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("assem20 - Sheet1", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            boolstatus = swModel.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swShowZoneLines, 0, true);
            boolstatus = swModel.Extension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swRevisionTableMultipleSheet

            if ((swDraw == null))
            {
                Debug.Print(" Please open a drawing document. ");
            }

            currentsheet = (Sheet)swDraw.GetCurrentSheet();
            swDraw.ActivateSheet(currentsheet.GetName());

            // Create sheet, Test, with 4 zones
            boolstatus = swDraw.NewSheet4("Test", (int)swDwgPaperSizes_e.swDwgPaperAsize, (int)swDwgTemplates_e.swDwgTemplateAsize,
            0.5, 0.5, 0.5, 0.5, 2, 2);

            System.Diagnostics.Debugger.Break();

            boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 0, 0, 0, false, 0, null, 0);
            swDraw.EditTemplate();
            swModel.EditSketch();
            swModel.ClearSelection2(true);
            boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 0.0812585524728589, 0.139959974668275, 0, false, 0,

            // Modify Test to have 9 zones
            boolstatus = swDraw.SetupSheet6("Test", (int)swDwgPaperSizes_e.swDwgPapersUserDefined, (int)swDwgTemplates_e.swDwgTempl
            landscape.slddrt", 0.2794, 0.2159, "Default",
            false, 0.5, 0.5, 0.5, 0.5, 3, 3);

            swDraw.EditSheet();
            swModel.EditSketch();
            swModel.ForceRebuild3(true);

            currentsheet = (Sheet)swDraw.GetCurrentSheet();
            swDraw.ActivateSheet(currentsheet.GetName());

            // Insert a revision table and add a revision row
            revTableAnno = currentsheet.InsertRevisionTable2(true, 0.0, 0.0, (int)swBOMConfigurationAnchorType_e.swBOMConfiguration/
            Debug.Print("Revision table annotation");
```

```

        Debug.Print(" New revision: " + revTableAnno.AddRevision("A"));
        Debug.Print(" Current revision: " + revTableAnno.CurrentRevision);

        RevisionTableFeature revTableFeat = default(RevisionTableFeature);
        revTableFeat = revTableAnno.RevisionTableFeature;
        Debug.Print("Revision table feature");
        Debug.Print(" Number of revision table annotations: " + revTableFeat.GetTableAnnotationCount());

        Feature feat = default(Feature);
        feat = (Feature)revTableFeat.GetFeature();
        Debug.Print("Feature: " + feat.Name);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Create Drawing Sheet Zones Example (VBA)

This example shows how to create a drawing sheet with zones, modify the zones in the drawing sheet, and insert a revision table.

```

' -----
' Preconditions:
' 1. Verify that the specified model document and templates exist.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new sheet named Test with four zones.
' 2. Inspect the graphics area.
' 3. Press F5.
' 4. Modifies Test to contain nine zones.
' 5. Creates Revision Table1.
' 6. Adds a revision row to the table.
' 7. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
' -----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swDraw As SldWorks.DrawingDoc
Dim currentsheet As SldWorks.Sheet
Dim swModel As SldWorks.ModelDoc2
Dim revTableAnno As SldWorks.RevisionTableAnnotation
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw", 3, 0,
    "", longstatus, longwarnings)
    swApp.ActivateDoc2 "assem20 - Sheet1", False, longstatus
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    boolstatus = swModel.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swShowZoneLines, 0, True)
    boolstatus = swModel.Extension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swRevisionTableMultipleSheetStyle, 0,
    swRevisionTableMultipleSheetStyle_e.swRevisionTable_Independent)

    If (swDraw Is Nothing) Then
        MsgBox " Please open a drawing document. "
        End
    End If

    Set currentsheet = swDraw.GetCurrentSheet
    swDraw.ActivateSheet (currentsheet.GetName)

    ' Create sheet, Test, with 4 zones
    boolstatus = swDraw.NewSheet4("Test", swDwgPaperASize, swDwgTemplateASize, 1, 1, True, "", 0, 0, "", 0.5, 0.5, 0.5, 0.5, 2, 2)

    Stop

    boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditTemplate
    swModel.EditSketch
    swModel.ClearSelection2 True
    boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 8.12585524728589E-02, 0.139959974668275, 0, False, 0,
    Nothing, 0)

    ' Modify Test to have 9 zones
    boolstatus = swDraw.SetupSheet6("Test", swDwgPapersUserDefined, swDwgTemplateCustom, 1, 1, True, "C:\Program Files\SOLIDWORKS
    Corp\SOLIDWORKS\lang\English\sheetformat\A - landscape.slddrt", 0.2794, 0.2159, "Default", False, 0.5, 0.5, 0.5, 0.5, 3, 3)

    swModel.EditSheet
    swModel.EditSketch
    swModel.ForceRebuild3 True

    Set currentsheet = swDraw.GetCurrentSheet
    swDraw.ActivateSheet (currentsheet.GetName)

    ' Insert a revision table and add a revision row
    Set revTableAnno = currentsheet.InsertRevisionTable2(True, 0#, 0#, swBOMConfigurationAnchor_TopLeft, "C:\Program
    Files\SOLIDWORKS Corp\SOLIDWORKS\lang\English\standard revision block.sldrevtbl", swRevisionTable_CircleSymbol, True)
    Debug.Print "Revision table annotation"
    Debug.Print " New revision: " & revTableAnno.AddRevision("A")
    Debug.Print " Current revision: " & revTableAnno.CurrentRevision

    Dim revTableFeat As SldWorks.RevisionTableFeature
    Set revTableFeat = revTableAnno.RevisionTableFeature

```

```
Debug.Print "Revision table feature"  
Debug.Print "  Number of revision table annotations: " & revTableFeat.GetTableAnnotationCount
```

```
Dim feat As SldWorks.Feature  
Set feat = revTableFeat.GetFeature  
Debug.Print "Feature: " & feat.Name
```

```
End Sub
```

Create Drawing Sheet Zones Example (VB.NET)

This example shows how to create a drawing sheet with zones, modify the zones in the drawing sheet, and insert a revision table.

```

'-----
' Preconditions:
' 1. Verify that the specified model document and templates exist.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new sheet named Test with four zones.
' 2. Inspect the graphics area.
' 3. Press F5.
' 4. Modifies Test to contain nine zones.
' 5. Creates Revision Table1.
' 6. Adds a revision row to the table.
' 7. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swDraw As DrawingDoc
    Dim currentsheet As Sheet
    Dim swModel As ModelDoc2
    Dim revTableAnno As RevisionTableAnnotation
    Dim boolstatus As Boolean
    Dim longstatus As Integer, longwarnings As Integer

    Sub main()

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\assem20.slddrw", 3, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("assem20 - Sheet1", False, longstatus)
        swModel = swApp.ActiveDoc
        swDraw = swModel

        boolstatus = swModel.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swShowZoneLines, 0, True)
        boolstatus = swModel.Extension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swRevisionTableMultipleSheetStyle, 0,

        If (swDraw Is Nothing) Then
            MsgBox(" Please open a drawing document. ")
        End If

        currentsheet = swDraw.GetCurrentSheet
        swDraw.ActivateSheet(currentsheet.GetName)

        ' Create sheet, Test, with 4 zones
        boolstatus = swDraw.NewSheet4("Test", swDwgPaperSizes_e.swDwgPaperASize, swDwgTemplates_e.swDwgTemplateASize, 1, 1, True, ""

        Stop

        boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditTemplate()
        swModel.EditSketch()
        swModel.ClearSelection2(True)
        boolstatus = swModel.Extension.SelectByID2("Sheet Format2", "SHEET", 0.0812585524728589, 0.139959974668275, 0, False, 0, Not

        ' Modify Test to have 9 zones
        boolstatus = swDraw.SetupSheet6("Test", swDwgPaperSizes_e.swDwgPapersUserDefined, swDwgTemplates_e.swDwgTemplateCustom, 1, 1
landscape.slddrt", 0.2794, 0.2159, "Default", False, 0.5, 0.5, 0.5, 3, 3)

        swModel.EditSheet()
        swModel.EditSketch()
        swModel.ForceRebuild3(True)

        currentsheet = swDraw.GetCurrentSheet
        swDraw.ActivateSheet(currentsheet.GetName)

        ' Insert a revision table and add a revision row
        revTableAnno = currentsheet.InsertRevisionTable2(True, 0.0#, 0.0#, swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_1
        Debug.Print("Revision table annotation")
        Debug.Print(" New revision: " & revTableAnno.AddRevision("A"))
        Debug.Print(" Current revision: " & revTableAnno.CurrentRevision)

        Dim revTableFeat As RevisionTableFeature
        revTableFeat = revTableAnno.RevisionTableFeature
        Debug.Print("Revision table feature")
        Debug.Print(" Number of revision table annotations: " & revTableFeat.GetTableAnnotationCount)

        Dim feat As Feature
        feat = revTableFeat.GetFeature
    
```

```
    Debug.Print("Feature: " & feat.Name)
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```